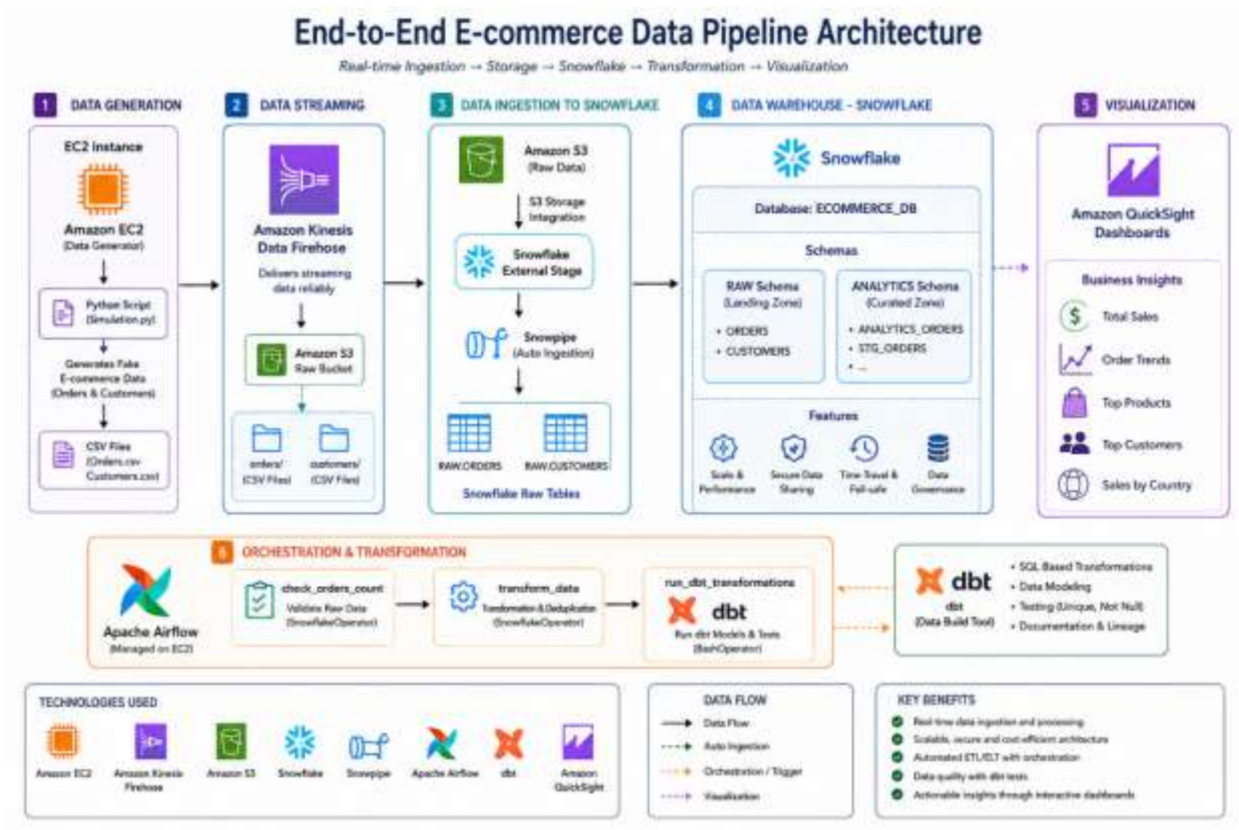
Ecommerce Data Pipeline using Snowflake and Managed Airflow on AWS



Ingestion (Kinesis)

↓

Storage (S3)

↓

Transformation (Snowflake)

↓

Orchestration (Airflow)

PHASE 1: DATA INGESTION

Step 1: IAM Role setup

Ye samajh lijiye ke aap apne EC2 server ko ek "ID Card" de rahe hain taaki woh doosri AWS services (Kinesis aur S3) istemal kar sake.

1. IAM Dashboard mein Jayen

- AWS Console ke search bar mein **IAM** likh kar open karein.
- Left side menu mein **Roles** par click karein.
- Blue button **Create role** par click karein.

2. Trusted Entity Select Karein

- **Trusted entity type:** AWS service
- **Service or use case:** EC2 (Q k ye role hum EC2 instance ko denge).
- Niche se dobara EC2 select karke Next par click karein.

3. Permissions (Policies) Attach Karein

Ab "Add permissions" wala page aayega. Yahan search bar mein ek-ek karke ye 3 naam likhein aur unke saath wale **checkbox ko tick** karein:

AmazonKinesisFirehoseFullAccess: Ye agent ko data pipe (Firehose) mein bhejne ki ijazat dega.
CloudWatchFullAccess: Iske baghair agent logs mein "Access Denied" ka error deta rahega.
AmazonS3FullAccess: Taaki Firehose aapki bucket mein file likh sake.

*Teeno select karne ke baad **Next** par click karein.*

4. Role ka Naam aur Finish

- **Role name:** Kinesis-Agent-Role-Ecommerce
- Sab check kar lein ke teeno policies list mein nazar aa rahi hain.
- Niche **Create role** par click kar dein.

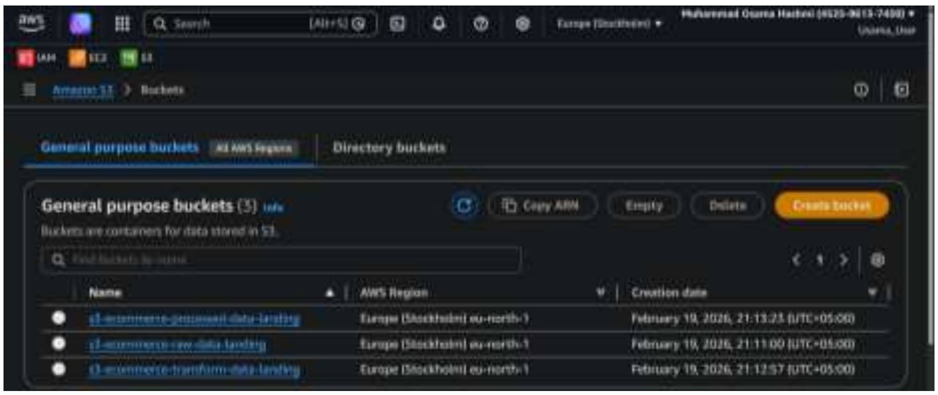
Step 2: S3 Buckets & folders (The Storage)

Create Buckets:

Hamen data rakhne ke liye 3 alag stages (buckets) chahiye hongi.

1. AWS Console k search bar mein **S3** likhein aur open karein.
2. **Create bucket** ke orange button par click karein.
3. Humein ek ek karke ye **3 buckets** banani hain:

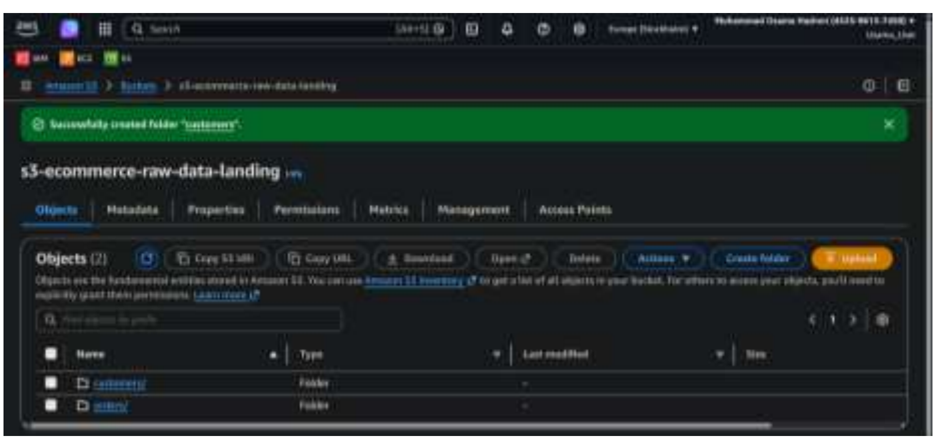
Bucket 1 (RAW Data): s3-ecommerce-raw-data-landing *(Isme Kinesis se raw data aayega)*
Bucket 2 (Transform Data): s3-ecommerce-transform-data-landing *(Isme Snowflake saaf data rkhega)*
Bucket 3 (Processed Data): s3-ecommerce-processed-data-landing *(Ye final data hoga QuickSight k liye)*



Create Folders in RAW Bucket

Kinesis Firehose data ko alag alag karne ke liye humein RAW bucket mein folders chahiye honge.

1. Apni raw bucket (s3-ecommerce-raw-data-landing) k naam pr click kr k use open karen.
2. **Create folder** par click karein.
3. Pehle folder ka naam likhein: orders aur create kar dein.
4. Dobara **Create folder** par click karein.
5. Doosre folder ka naam likhein: customers aur create kar dein.



Step 3: Kinesis Data Firehose Streams

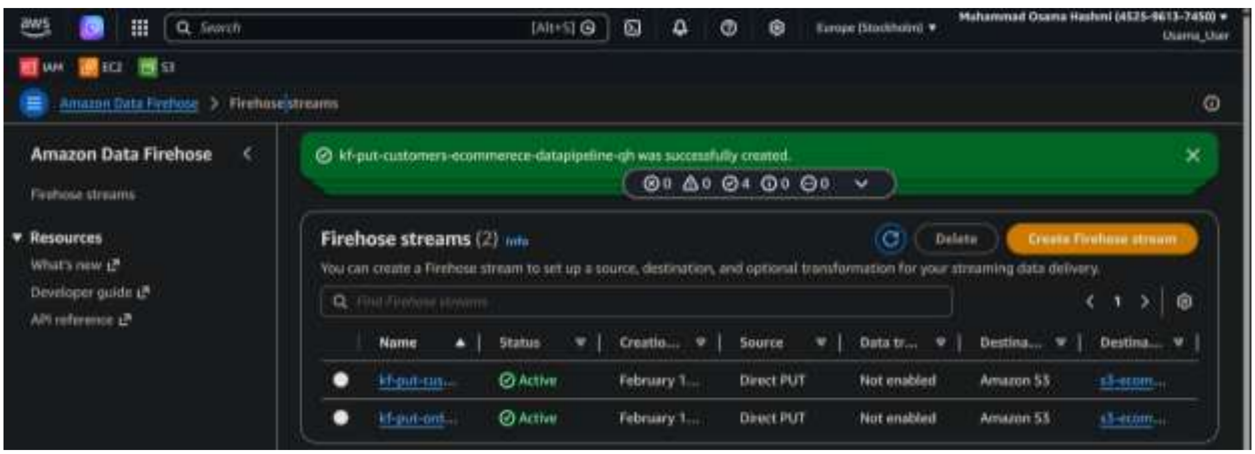
Create (2) Kinesis Data Firehose Streams

(For Orders)

1. AWS Console mein **Kinesis** search kar ke open karein.
2. **Amazon Data Firehose → Create Firehose stream** pr click karein.
    - **Source:** Direct PUT
    - **Destination:** Amazon S3 select
    - **Firehose stream name:** kf-put-orders-ecommerce-datapipeline-oh
    - **S3 bucket:** Browse se s3-ecommerce-raw-data-landing choose kren
    - **S3 bucket prefix:** orders/
    - **Create Firehose stream** par click kar den.

(For Customers)

- Bilkul oopar wala process repeat karein, lekin is baar:
    - **Firehose stream name:** kf-put-customers-ecommerce-datapipeline-oh
    - **S3 bucket prefix:** customers/



Ab hamari "Pipe" (Firehose) aur "Storage" (S3) dono tayyar hain. Ab hamein woh Source banani hai jahan se data niklega—yani aapka EC2 Server.

Step 4: EC2 Instance Launch Karna

Now create a new Linux server aur usse woh "ID Card" (IAM Role) denge jo main ne banaya tha.

1. AWS Search bar mein EC2 likhein aur Dashboard par jayen.
2. Launch instance par click karen.
3. **Name:** Ecommerce-Data-Server
4. **Application and OS Images (AMI):** Amazon Linux 2023 kernel-6.1 AMI
5. **Instance type:** t3.medium (Free tier eligible) select karen.
6. **Key pair:** Create new key pair par click karein via .pem file download karein.
    - **Key pair name:** <your-key>
    - **Key pair type:** RSA
    - **Private key file format:** Agar aap Windows use kar rahe hain aur VS Code terminal se connect karenge, toh .pem select karein.
    - Create key pair par click karte hi ek file download hogi.
      Is File ko C:\Users\STARIZ.PK\Desktop\CDE\Projects\Ecommerce_Project Folder mein cut/paste kr den
7. **Network settings:** Allow SSH traffic from My IP
8. **Advanced details:**
    - **IAM instance profile:** Select your role Kinesis-Agent-Role-Ecommerce

    Ab Launch instance par click kar den.

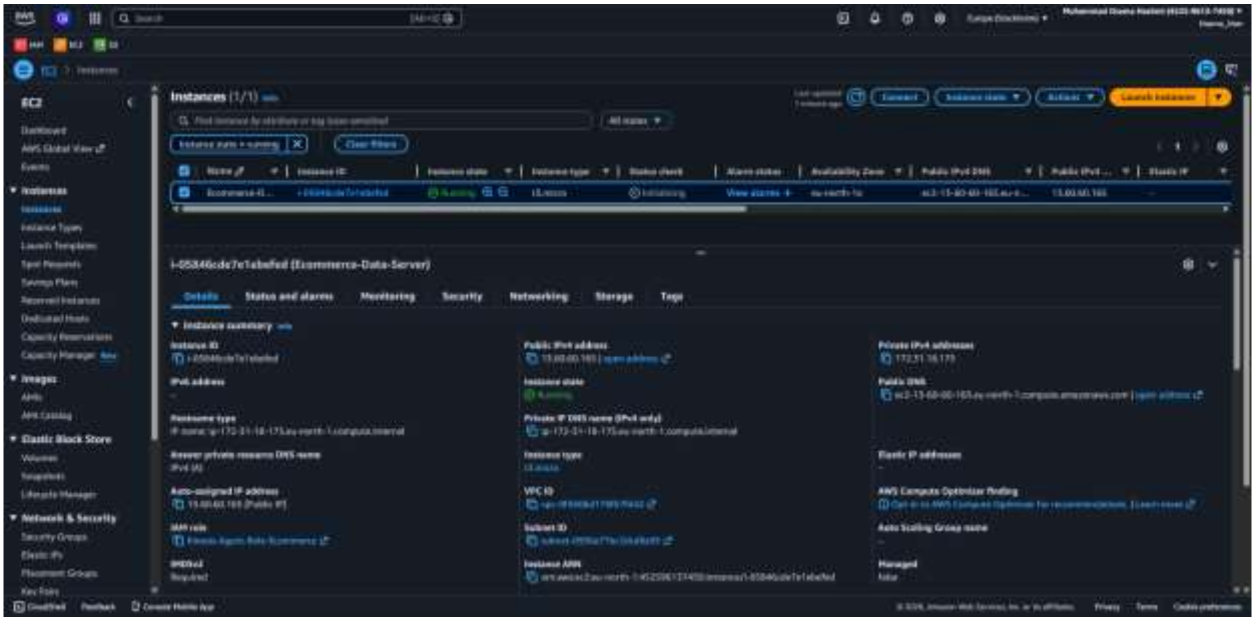
Now **EC2 Machine** (Ecommerce-Data-Server) is Ready.

Step 5: EC2 se Connect aur Agent Install Karna

Jb instance "**Running**" state mein a jae, to apne VS Code terminal se **SSH** ke zrye connect kren:

1) EC2 se Connect karein (VS Code Terminal se)

1. Apna Public IP Address ko copy kren
    - *Apne **AWS Console** mein jayen aur **EC2 Dashboard** par click karein.*
    - ***Instances (running)** par jaen aur apne banae hue running instance pr tick kren.*
    - *Niche "Details" tab mein **Public IPv4 address** (e.g., <your-public-ip>) copy karen.*



2. Apna **Visual Studio Code** *(New window)* open karen
3. Click on **File** -> **Open Folder** -> open in this folder ***D:\AWS_SF_AF_Pipeline_Project***
4. Click on **Terminal** -> ***New Terminal***

Ab ye command likhein

```
# SSH connect karne k liye

ssh -i "<your-key>.pem" ec2-user@<your-public-ip>
```

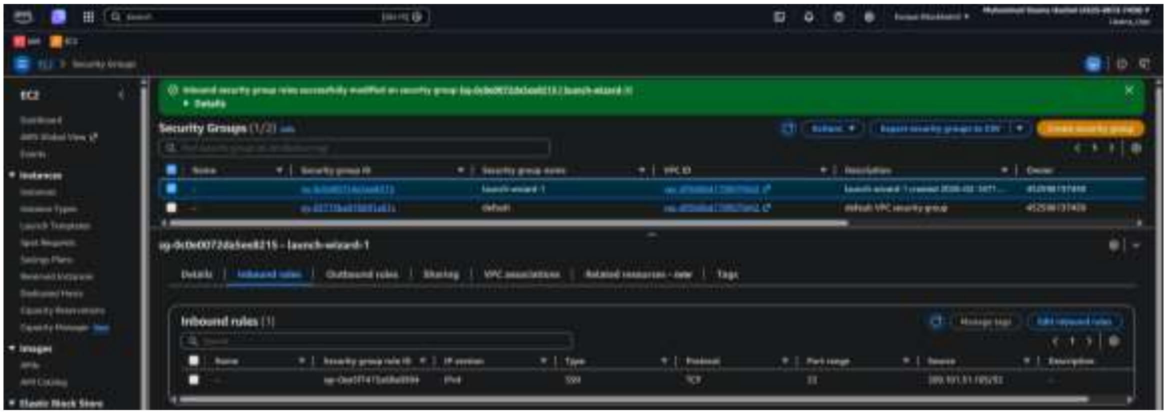
***Note:** Agr "Are you sure you want to continue connecting?" pooche.*

Solution: *Toh yes likh kar Enter daba den.*

***Note:** Agr ye likha ae "ssh: connect to host* <your-public-ip> *port 22: Connection timed out"*

Solution: AWS Console mein default security group kabhi kabhi SSH allow nahi karta.

- **AWS Console** mein jayen -> **EC2 Instances**.
- Apne instance par click karein aur niche **Security** tab mein jayen.
- Jo Security Group wahan likha hai (e.g., launch-wizard-1), us par click karein.
- **Inbound Rules** check karein. Kya wahan **SSH** (Port 22) add hai?
- Agar nahi, toh **Edit inbound rules** par click karein aur ye add karein:
  - **Type:** SSH
  - **Protocol:** TCP
  - **Port Range:** 22
  - **Source:** My IP
  - **Click on** Save rules



Note: Agr ye error aa raha ho (**"Permissions for '<your-key>.pem' are too open"**), yeh Windows par bohot common hai. SSH security k liye zaroori hai k aapki **.pem** file sirf aap (Current User) hi parh saken, koi aur nahi.

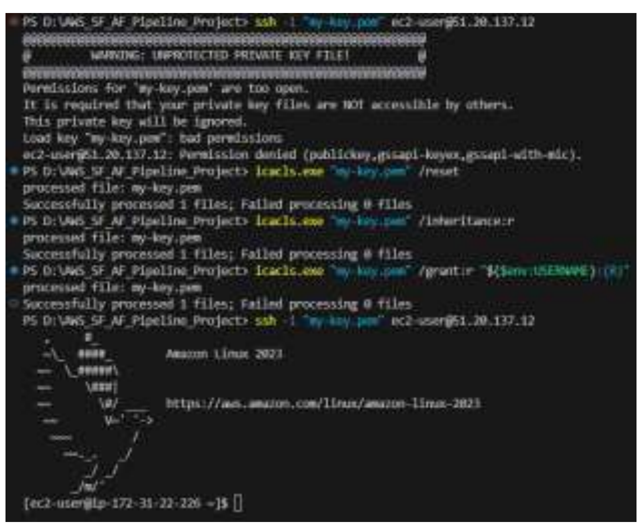
Solution: Isse fix karne ke liye humein is file ki **Security Settings** sahi karni hogi.

Apne VS Code terminal (PowerShell) mein niche di gyi commands ko **one by one** chalaen.

1. **Permissions ko Reset krne ke liye:**
   icacls.exe "<your-key>.pem" /reset
2. **File ki Inheritance khatam krne ke liye:**
   icacls.exe "<your-key>.pem" /inheritance:r
3. **Sirf apne user ko "Read" permission dene ke liye:**
   icacls.exe "<your-key>.pem" /grant:r "$($env:USERNAME):(R)"

Ab Dubara Connect Karein:

ssh -i "<your-key>.pem" ec2-user@<your-public-ip>



---

2) Kinesis Agent Install karein

Ek baar login hone ke baad, humein **Kinesis Agent** install karna hai jo files par nazar rakhega:

```
# 1. System ko update karein
sudo yum update -y

# 2. Kinesis Agent install karein
sudo yum install -y aws-kinesis-agent
```

Step 6: Agent Configuration

Hum /etc/aws-kinesis/agent.json file ko edit karenge. Is file mein hum batate hain ke server par file kahan par hai aur use kis Firehose pipe mein bhejna hai.

1. **Configuration file kholen:**

```
sudo nano /etc/aws-kinesis/agent.json
```

2. **Purana saara text mita dein aur ye niche wala code copy-paste karein:** *(Yaqeen karlein ke deliveryStream ke naam bilkul wahi hain jo aapne AWS console mein banaye hain)*.

```
{
 "cloudwatch.emitMetrics": true,
 "firehose.endpoint": "firehose.eu-north-1.amazonaws.com",
 "flows": [
  {
   "filePattern": "/tmp/orders.csv*",
   "deliveryStream": "kf-put-orders-ecommerce-datapipeline-oh",
   "initialPosition": "START_OF_FILE"
  },
  {
   "filePattern": "/tmp/customers.csv*",
   "deliveryStream": "kf-put-customers-ecommerce-datapipeline-oh",
   "initialPosition": "START_OF_FILE"
  }
 ]
}
```

Note: *Agar aapka region eu-north-1 (Stockholm) nahi hai, toh use apne region ke mutabiq change karlein.*

Save and Exit: Ctrl + O dabayein, phir Enter (For Save), then Ctrl + X (For Exit)

Step 7: Agent ko Start aur Enable Karna

Ab hum agent ko batayenge ke kaam shuru kare aur computer restart hone par bhi khud hi chalu ho jaye.

```
# Agent ko chalu karein
sudo service aws-kinesis-agent start

# Ye lazmi karein taaki agent hamesha chalta rahe
sudo chkconfig aws-kinesis-agent on
```

Step 8: Permissions ka "Jadu" (Folders setup)

Agent ko files parhne ke liye ijazat chahiye hoti hai. Ye commands terminal mein chalaein:

```
sudo chmod 777 /tmp
```

Ab hamara pura system **(EC2 -> Agent -> Firehose -> S3)** jur chuka hai. Lekin abhi /tmp/ folder mein koi data nahi hai, isliye S3 mein kuch nahi ja raha.

Hamen ab ek **Simulation Script (Python)** chahiye jo hr second fake orders banae aur /tmp/orders.csv aur /tmp/customers.csv mein likhe.

Iske liye humein EC2 par Python aur kuch libraries install karni hongi.

Pehle humein EC2 machine ko Python ke liye taiyar karna hoga.

Step 9: Python Environment Setup

EC2 terminal mein ye commands chalaein:

1. **Python install karein:**

```
sudo yum install python3 -y
```

2. **Faker library install karein** (Ye fake names, addresses aur orders banane me kam ati hai):

   Pehle humein `pip` ko install karna hoga ta k hum `Faker` library download kar sakein:

```
sudo yum install python3-pip -y
```

   Faker Library Install Karein

```
pip3 install Faker
```

Step 10: Simulation Script Likhna

Ab hum ek file banainge jisme hamara logic hoga.

1. **File create karen:**

```
nano simulation.py
```

2. **Niche diya gaya code copy karke paste karein:** *(Ye script har 2 second baad ek naya order aur customer generate karke /tmp/ folder mein save karegi)*.

```
import csv
import time
from faker import Faker
import random

fake = Faker()

def generate_data():
  while True:
    # Orders Data
    order_data = [random.randint(1000, 9999), fake.name(), random.choice(['Laptop', 'Phone', 'Watch']), random.randint(50, 500)]
    with open('/tmp/orders.csv', 'a') as f:
      writer = csv.writer(f)
      writer.writerow(order_data)

    # Customers Data
    customer_data = [fake.name(), fake.email(), fake.country()]
    with open('/tmp/customers.csv', 'a') as f:
      writer = csv.writer(f)
      writer.writerow(customer_data)

    print(f"Generated: Order for {order_data[1]} and Customer {customer_data[0]}")
    time.sleep(2) # Har 2 second baad data banega

if __name__ == "__main__":
  generate_data()
```

Save and Exit: Ctrl + O, Enter, phir Ctrl + X.

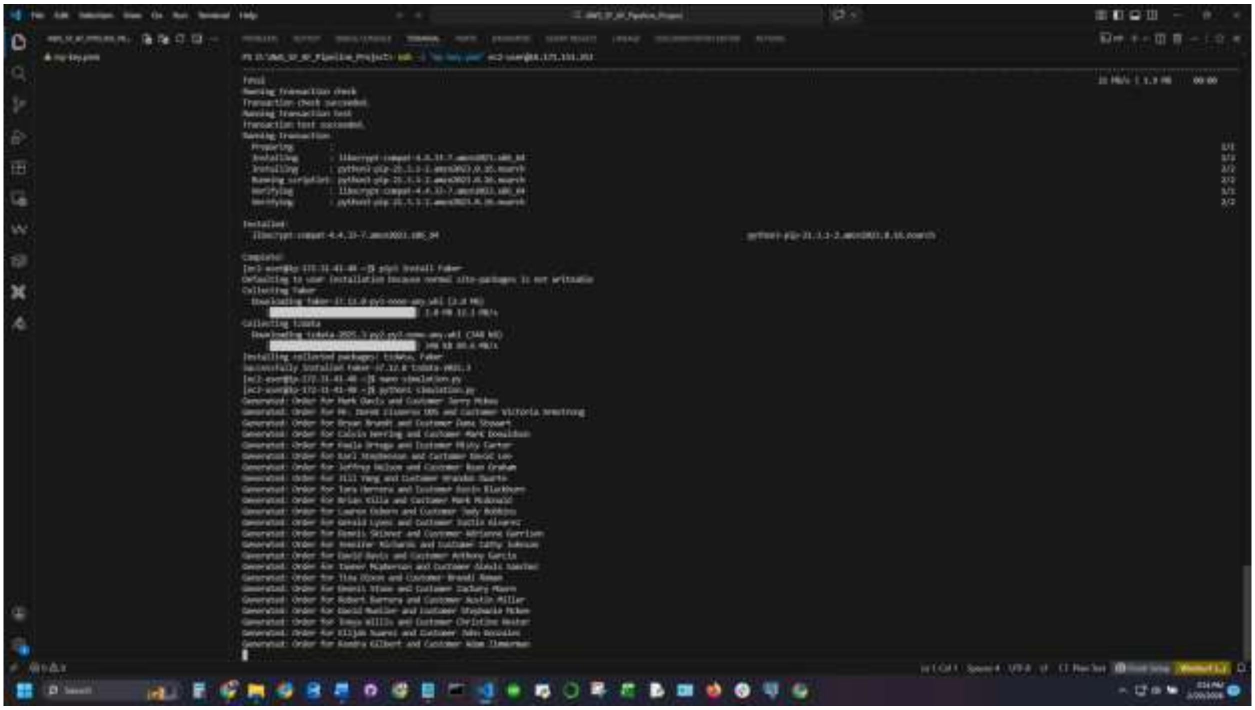
Step 11: The Grand Finale (Data Pipe running)

Ab waqt hai results dekhne ka

Script chalu karein:

```
python3 simulation.py
```

(Ab aapko screen par "Generated: Order for..." nazar aana shuru ho jayega).



Ab hamen ye check krna hai k kya **Kinesis Agent** is data ko utha kr AWS tk pohcha rha hai ya nhi.

Step 12: New Terminal se EC2 mein Login karein

Pehle new terminal mein SSH command chala kar dobara EC2 ke andar dakhil hon:

```
ssh -i "<your-key>.pem" ec2-user@<your-public-ip>
```

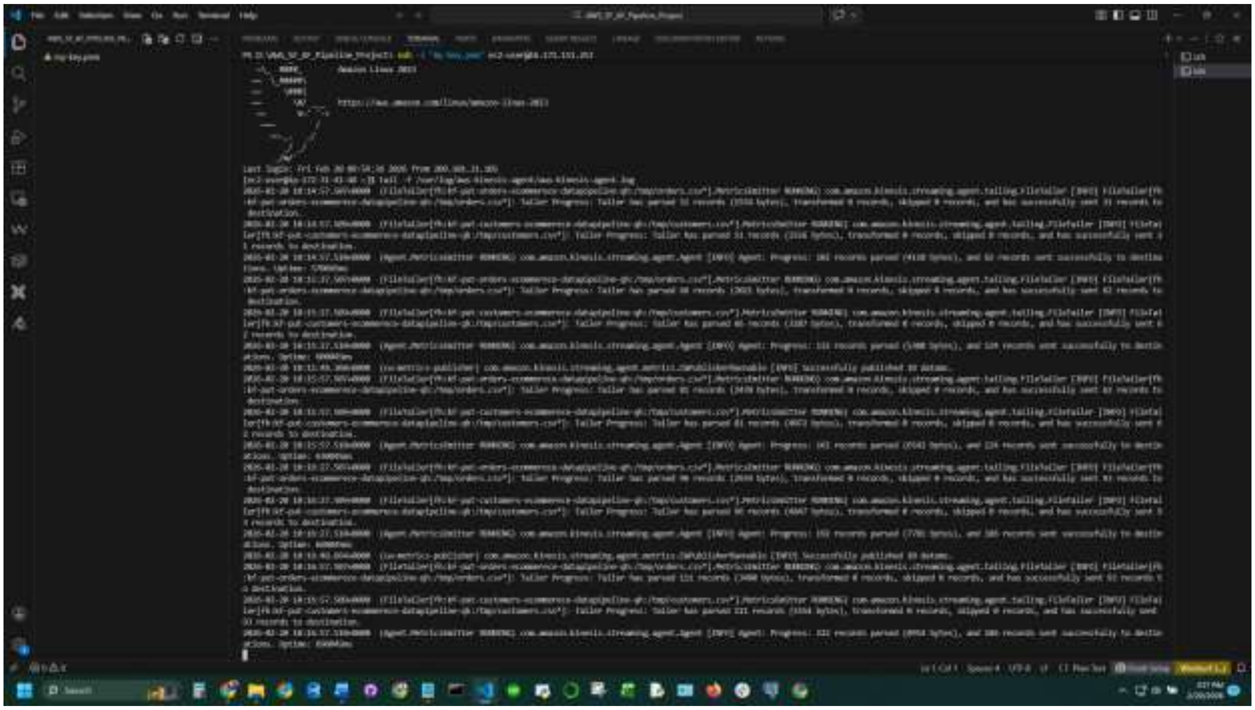
Step 13: Agent ke Logs Check karen

Jab aap EC2 ke andar pohonch toh ye command chalaein:

```
tail -f /var/log/aws-kinesis-agent/aws-kinesis-agent.log
```

Result Check

1. **Successfully sent records:** Agar aapko nazar aaye Successfully sent 1 records to destination, toh iska matlab hai pipe 100% kaam kar rahi hai.
2. **Errors:** Agar AccessDenied ya SubscriptionRequiredException nazar aaye, toh batayein (waise Root upgrade ke baad ye nahi aana chahiye).
3. **File Tailer:** Agar likha aaye Tailing file: /tmp/orders.csv, toh iska matlab hai agent file ko parh raha hai.



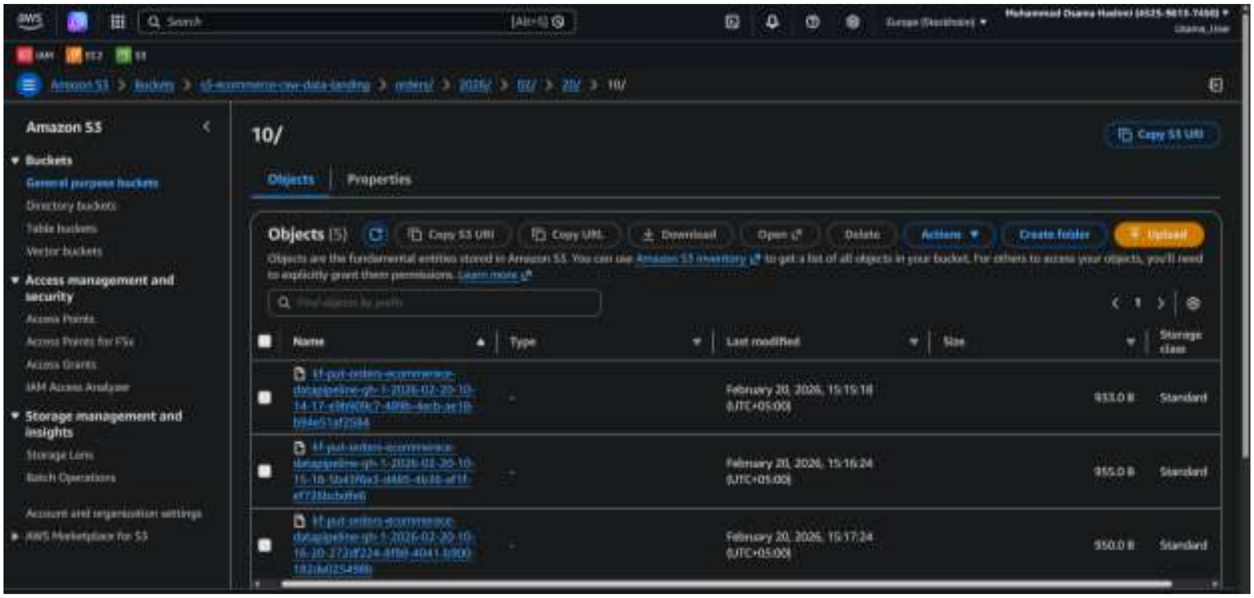
Mubarak ho! Apki mehnt rng lai. Apka Phase 1 (Data Ingestion) bilkul **100% kamyab** ho gaya hai.

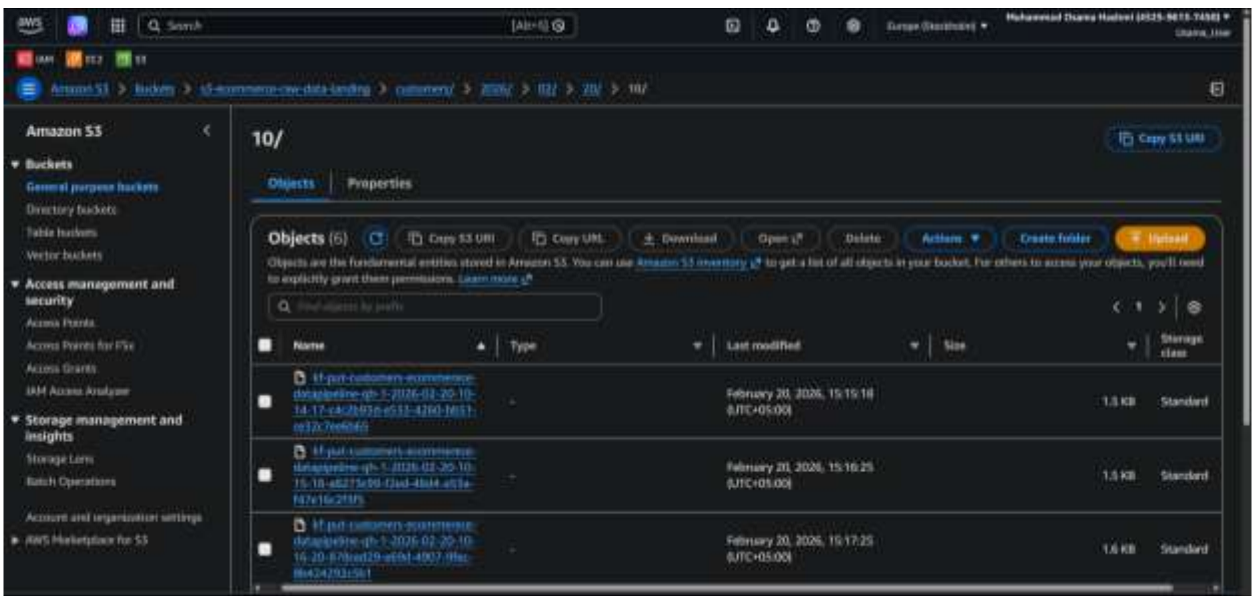
- **Tailer Progress:** Agent ne /tmp/orders.csv aur /tmp/customers.csv dono files ko parhna (tailing) shuru kar diya hai.
- **Success Records:** Logs mein baar baar likha aa raha hai "and has successfully sent 557 records to destination" aur "588 records sent successfully".
- **Active Status:** Aapka agent live hai aur data real-time mein pipe ke zariye bhej raha hai.

Step 14: Final Verification

Aage barhne se pehle, apni aankhon se S3 mein data dekh lein:

1. **S3 Console** mein jayen.
2. s3-ecommerce-raw-data-landing bucket ko kholen.
3. orders/ aur customers/ folders ke andar jayen. Wahan aapko year (2026), month, aur days ke hisab se folders milenge. Unke aakhir mein aapko Files milengi.





S3 mein file ka naam abhi kf-put-orders-…… hoga. Us file pr click kr k Download kr k check karein, usme random orders aur customers ka data hona chahiye.

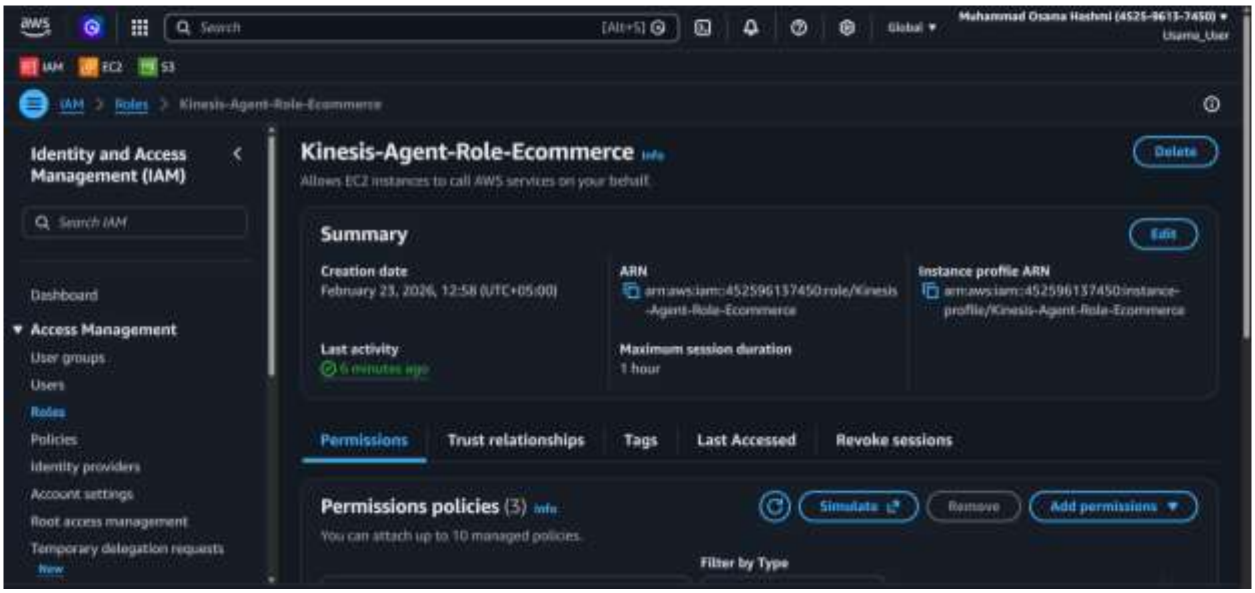
Congratulation! Aapka Phase 1 (Data Ingestion) mukammal ho gaya hai.

PHASE 2: SNOWFLAKE SETUP & DATA LOADING

Ab hum agle marhale ki taraf barhte hain jahan hum is S3 ke data ko Snowflake mein load karenge.

Copy ARN

- **AWS Console** mein login karein aur **IAM (Roles)** service par jayen.
- Apna banaya hua role dhundein: **Kinesis-Agent-Role-Ecommerce** aur us par click karein.
- Role khulte hi sbse upar **Summary** section mein aapko **ARN** nazar aayega use copy krlen

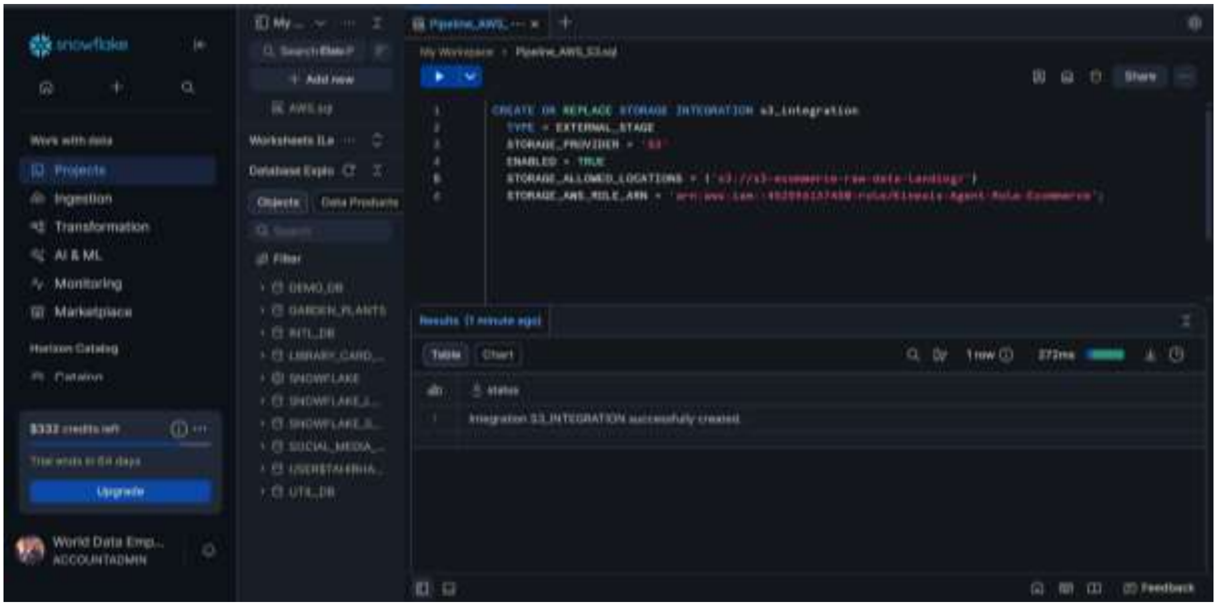


ARN: arn:aws:iam::452596137450:role/Kinesis-Agent-Role-Ecommerce

STEP 1: STORAGE INTEGRATION (SNOWFLAKE-S3 HANDSHAKE)

Snowflake mein ja kar ek new SQL Worksheet kholiye aur ye command chalaein. Isse Snowflake ko aapki S3 bucket tak access milegi:

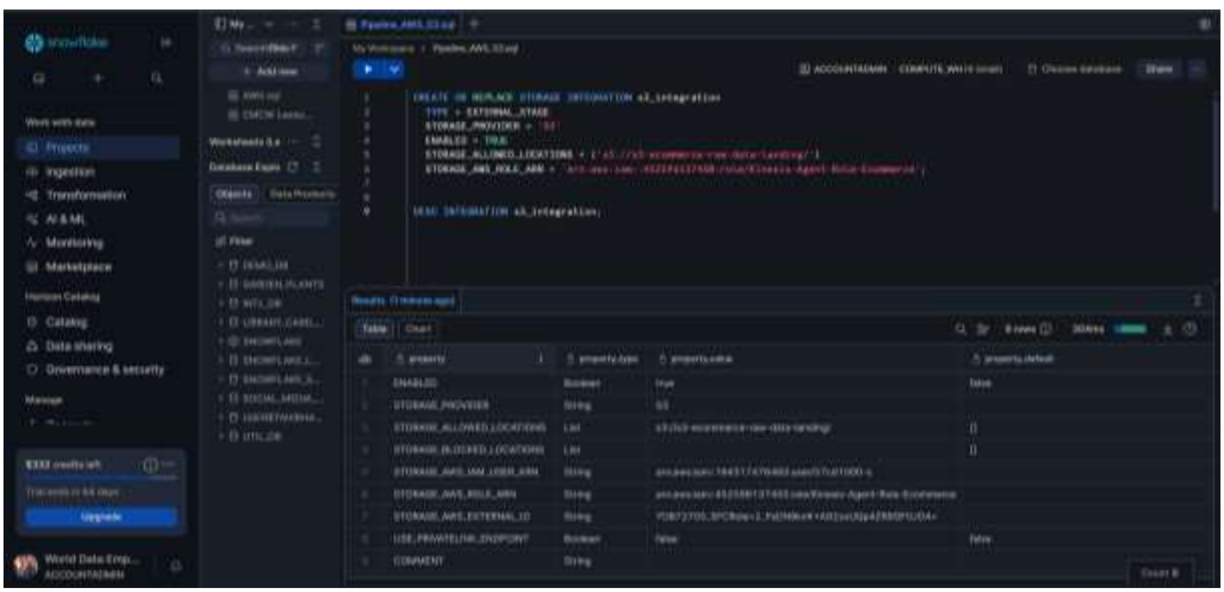
```
CREATE OR REPLACE STORAGE INTEGRATION s3_integration
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_ALLOWED_LOCATIONS = ('s3://s3-ecommerce-raw-data-landing/')
  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::452596137450:role/Kinesis-Agent-Role-Ecommerce';
```



STEP 2: CONNECTION DETAILS HASIL KAREIN

Upar wali command run karne ke baad, foran niche wali command chalaein:

```
DESC INTEGRATION s3_integration;
```



Note: Is command ke result mein aapko niche table mein do aham cheezein dikhengi:

1. **STORAGE_AWS_IAM_USER_ARN** (Ye Snowflake ka apna ARN hai).
   arn:aws:iam::194317476460:user/57cd1000-s

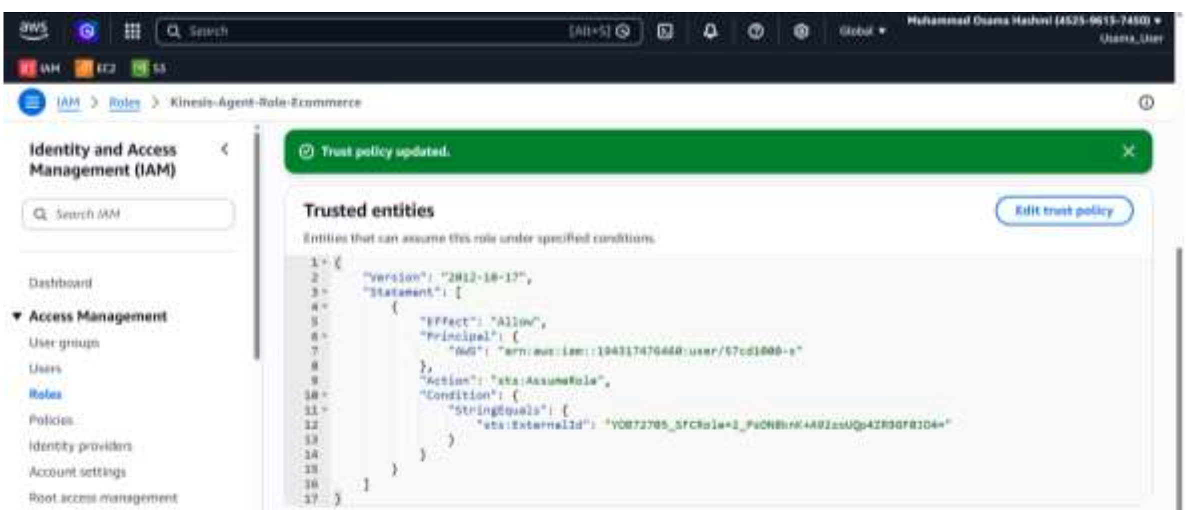
2. **STORAGE_AWS_EXTERNAL_ID** (Ye ek unique ID hai).
   YOB72705_SFCRole=2_8LsYDJOSoDmMib44sH4ClfWAleo=

STEP 3: AWS MEIN TRUST RELATIONSHIP UPDATE KAREIN

Ab aapko wapis AWS Console mein jana hai:

1. **IAM (Roles)** me jaen aur apna banaya hua Role (**Kinesis-Agent-Role-Ecommerce**) kholen.
2. **Trust Relationships** tab par click karein aur **Edit trust policy** par click karein.
3. Wahan purani policy ko hata kar ye niche wali policy paste karein (Lekin isme Snowflake se mili hui dono values update karen):
4. Update Policy pe krden

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Effect": "Allow",
   "Principal": {
    "AWS": "arn:aws:iam::194317476460:user/57cd1000-s"
   },
   "Action": "sts:AssumeRole",
   "Condition": {
    "StringEquals": {
     "sts:ExternalId": "YOB72705_SFCRole=4_aG8wCWUYcKIZFkG2/+bQB06iZ5Q="
    }
   }
  }
 ]
}
```



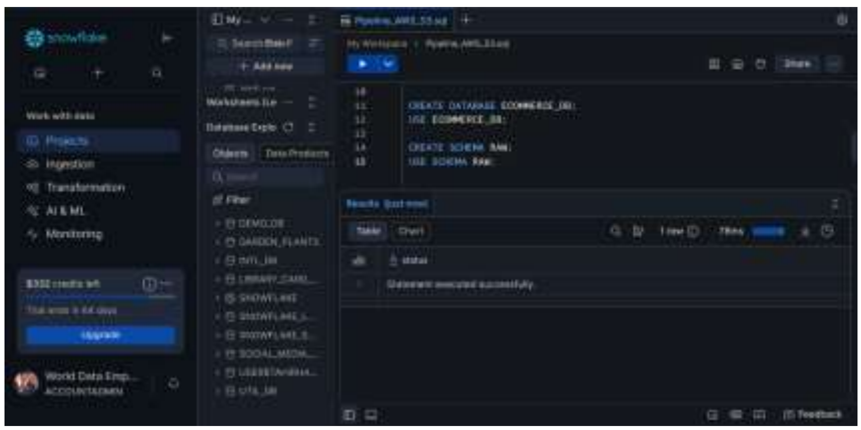
Snowflake mein **DESC INTEGRATION** run kar ke wo **(2)** values nikal k unhen AWS mein **update** karne ke baad hi Snowflake aapka data utha sakega.

Snowflake aur S3 ka "Handshake" mukammal ho chuka hai
Ab hum check karenge ke kya Snowflake waqai aapki bucket ko dekh pa raha hai.

1. Database aur Schema Context Set Karein

Pehle Snowflake ko batayein ke aapka database aur schema kahan hai.

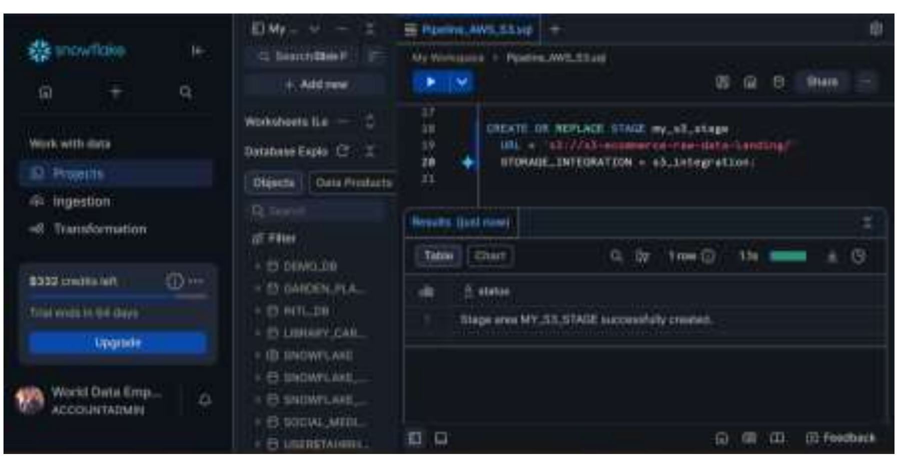
```
-- Database aur Schema select karein
CREATE DATABASE ECOMMERCE_DB;
USE ECOMMERCE_DB;

CREATE SCHEMA RAW;
USE SCHEMA RAW;
```



2. External Stage Banayein

Integration object ban gaya hai, ab humein ek **Stage** banana hai jo is integration ko use karega. Isse data load karna mazeed asan ho jayega:

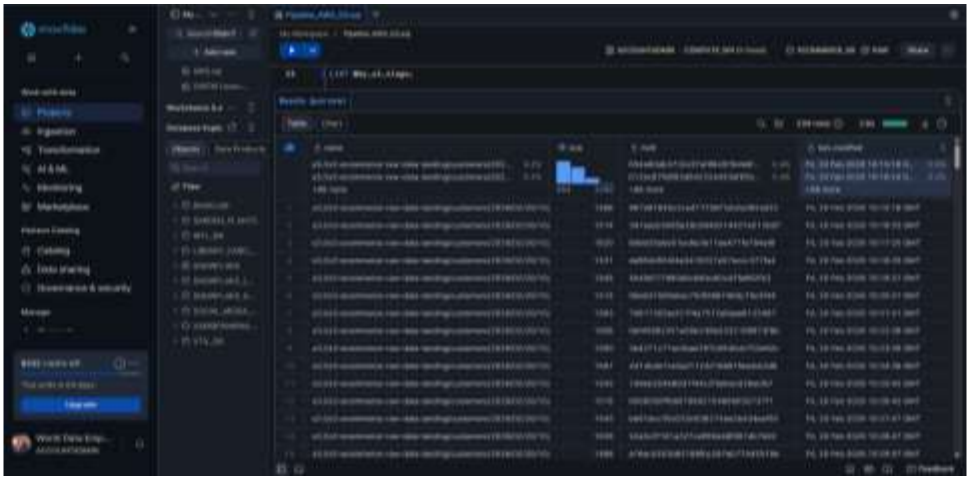
```
-- S3 ke liye Stage banana
CREATE OR REPLACE STAGE my_s3_stage
  URL = 's3://s3-ecommerce-raw-data-landing/'
  STORAGE_INTEGRATION = s3_integration;
```



3. Connection Test

Ab aap niche wali command chalayein, ye aapko S3 ki files dikhayegi:

```
LIST @my_s3_stage;
```



4. Create Tables for Orders & Customers

For Orders Table: Data format: ID, Name, Item, Price

```
CREATE OR REPLACE TABLE ECOMMERCE_DB.RAW.ORDERS (
    ORDER_ID NUMBER,
    CUSTOMER_NAME STRING,
    ITEM_NAME STRING,
    PRICE NUMBER
);
```

For Customers Table: Data format: Name, Email, Country

```
CREATE OR REPLACE TABLE ECOMMERCE_DB.RAW.CUSTOMERS (
    CUSTOMER_NAME STRING,
    EMAIL STRING,
    COUNTRY STRING
);
```

5. Snowpipe

Industry projects mein jab data real-time stream ho raha ho (jaisa ke aap ka Kinesis Firehose kar raha hai), toh **Snowpipe** hi behtareen solution hai.

Snowpipe Use Karne ke Faide:

1. **Automation:** Jese hi Firehose S3 me new file dalega, Snowpipe use forn detect krke Table mein load kar dega.
2. **Serverless:** Aap ko khud se command nahi chalani paregi, Snowflake backend par ise manage karega.
3. **Efficiency:** Ye choti-choti files ko load karne ke liye zyada behtar hai.

Snowpipe Setup Karne ka Tarika:

- **Step 1: Pipe Create Karein**

  Snowflake mein ye SQL commands run karein:

```sql
-- Orders ke liye Pipe
CREATE OR REPLACE PIPE ECOMMERCE_DB.RAW.ORDERS_PIPE
AUTO_INGEST = TRUE
AS
COPY INTO ECOMMERCE_DB.RAW.ORDERS
FROM @my_s3_stage/orders/
FILE_FORMAT = (TYPE = CSV FIELD_DELIMITER = ',' SKIP_HEADER = 0);

-- Customers ke liye Pipe
CREATE OR REPLACE PIPE ECOMMERCE_DB.RAW.CUSTOMERS_PIPE
AUTO_INGEST = TRUE
AS
COPY INTO ECOMMERCE_DB.RAW.CUSTOMERS
FROM @my_s3_stage/customers/
FILE_FORMAT = (TYPE = CSV FIELD_DELIMITER = ',' SKIP_HEADER = 0);
```

- **Step 2: S3 Event Notification (The Trigger)**

  Snowpipe ko btane k liye k "new file a gyi hai", hamen S3 pr ek (notification) lgana hoga:

1. Snowflake mein ye command chalaein: `SHOW PIPES`;
2. Wahan **notification_channel** column se lambi si ID (SQS QUEUE ARN) copy karein.

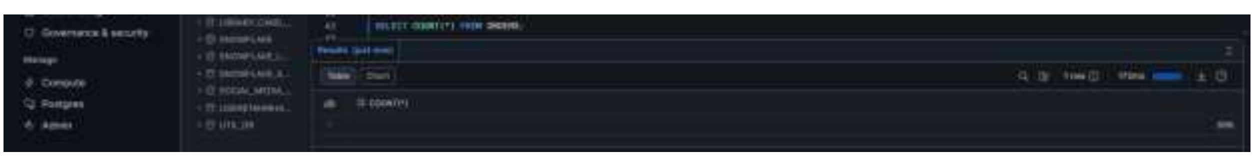
3. **AWS S3 Console** mein apni raw bucket par jaen -> **Properties** tab -> **Event notifications**.
4. "Create event notification" par click kren:
    - **Event name:** trigger_snowpipe_orders
    - **Prefix:** orders/
    - **Event types:** "All object create events"
    - **Destination:** "SQS Queue" select karein.
    - **SQS Queue ARN:** Snowflake se copy ki hui ID yahan paste kar den.
    - **Save Changes**
    - Same with customers
        - **Event name:** trigger_snowpipe_customers
        - **Prefix:** customers/

Check Point:

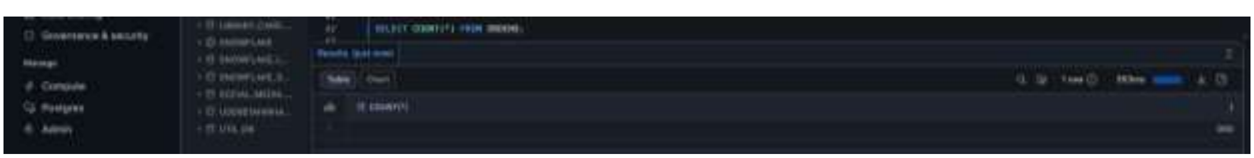
Jaise hi aap ye set karenge, aap ko EC2 terminal par data generate hote hue dekhna hai. Phir 1-2 minute baad Snowflake mein ye chala kar check karein:

```
SELECT COUNT(*) FROM ORDERS;
```

Aap dekhenge k numbers khud-ba-khud brh rahe honge jb aap phir se is command chalainge to!



After 1 min



PHASE 3: AIRFLOW SETUP

STEP 1: AIRFLOW DASHBOARD ON KAREN

Airflow ap k EC2 instance pr hi install hua hai, bs ab use Run kr k browser mein access krna hai. Apne EC2 terminal par jaen aur ye commands run karein:

1. Agr apka EC2 Terminal close ho gaya hai to open karlen is command se:

```
ssh -i "<your-key>.pem" ec2-user@<your-public-ip>
```

1. Airflow k liye Environment create aur activate kren

```
python3 -m venv airflow_env
source airflow_env/bin/activate
```

3. Installation of Airflow and Snowflake provider

```
pip install --upgrade pip

pip install "apache-airflow==2.8.1" apache-airflow-providers-snowflake --constraint
"https://raw.githubusercontent.com/apache/airflow/constraints-2.8.1/constraints-3.9.txt"
```

6. Airflow Home aur Database Setup

Home directory define karein

```
export AIRFLOW_HOME=~/airflow
```

Database initialize karen

```
airflow db init
```

7. Admin User Banaen (Dashboard Login k liye)

```
airflow users create \
  --username admin \
  --firstname Osama \
  --lastname Hashmi \
  --role Admin \
  --email osamahashmi28@gmail.com \
  --password 'Admin#123'
```

8. Webserver aur Scheduler Start Karen

Ab do new terminal tabs kholen (Har ek mein pehly ye 2 commands run kren

```
ssh -i "<your-key>.pem" ec2-user@<your-public-ip>

source airflow_env/bin/activate

export AIRFLOW_HOME=~/airflow
```

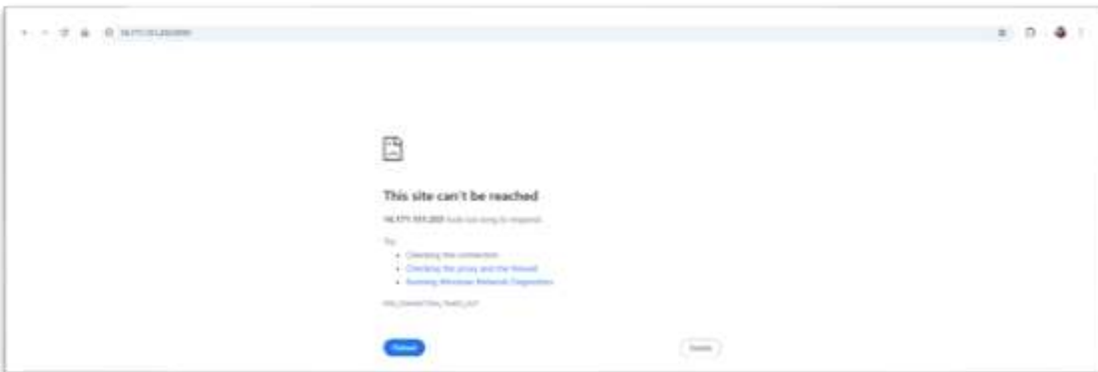
- **Terminal 1**: airflow scheduler -D
- **Terminal 2**: airflow webserver -D

DASHBOARD DEKHEN

Ab aap apne Chrome browser mein jaen, Ctrl + Shift + N press kren for Incognito mode aur ye type kr k enter kren browser mein:

```
http://<your-public-ip>:8080
```

Note: Agar page load na ho to:



Solution: ERR_CONNECTION_TIMED_OUT ka matlab aksar firewall hota hai.

1. AWS SECURITY GROUP MEIN PORT 8080 KHOLEN

1. **AWS Console** mein apne Instance par jayen.
2. **Security** tab -> **Security Groups** link par click karein.
3. **Inbound Rules** mein **Edit Inbound Rules** par click karen.
    - **"Add rule"** par click karein.
    - **Type:** Custom TCP
    - **Port range:** `8080`
    - **Source: "Anywhere-IPv4"** (`0.0.0.0/0`)
    - Niche **"Save rules"** par click kar dein.

2. Browser mein Dashboard Access Karein

Ab apne browser mein wapas jayen aur is page ko refresh karein:

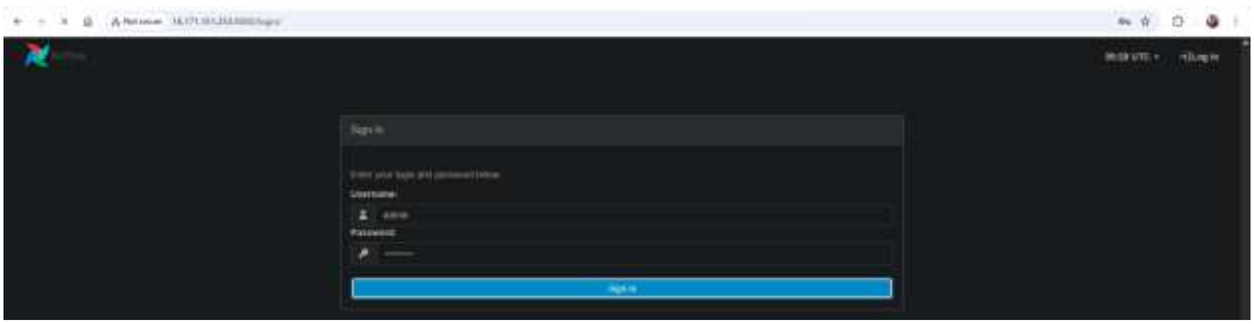
> http://<your-public-ip>:8080

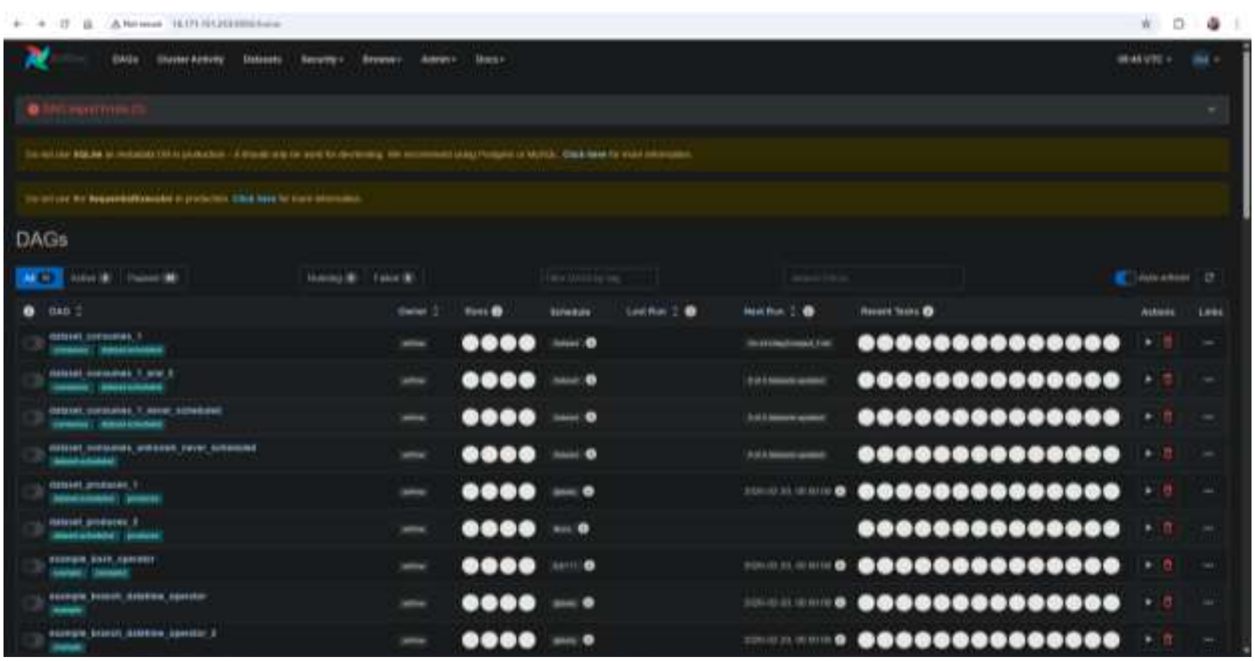
---

3. Sign In Karen

Jab page khul jaye, toh aapne jo user banaya tha usse login karein:

- **Username:** admin
- **Password:** Admin#123



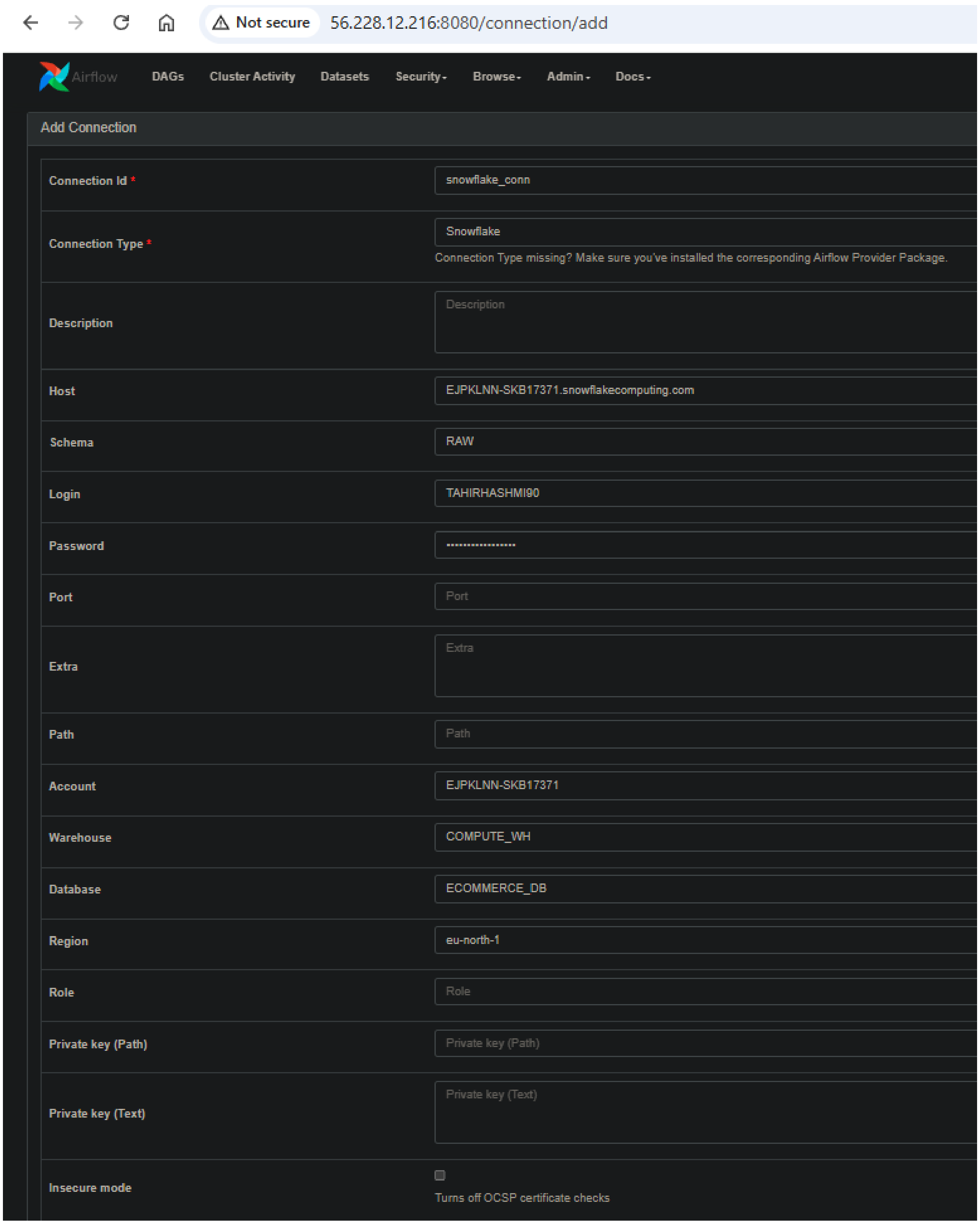


Ab ham **Transformation** k sabse ahem hisse par hain yahan par ek **DAG** banainge jo **"Continuous Transformation"** karega.. Initially hamein ***Airflow*** ko ***Snowflake*** se "baat" krwani hai.

STEP 1: SNOWFLAKE CONNECTION SETUP

Dashboard par upar menu mein arrow le k jayein aur ye path follow karein:

1 **Admin** > **Connections** par click karein.
2 New connection add karne ke liye **(+)** button dabaen.



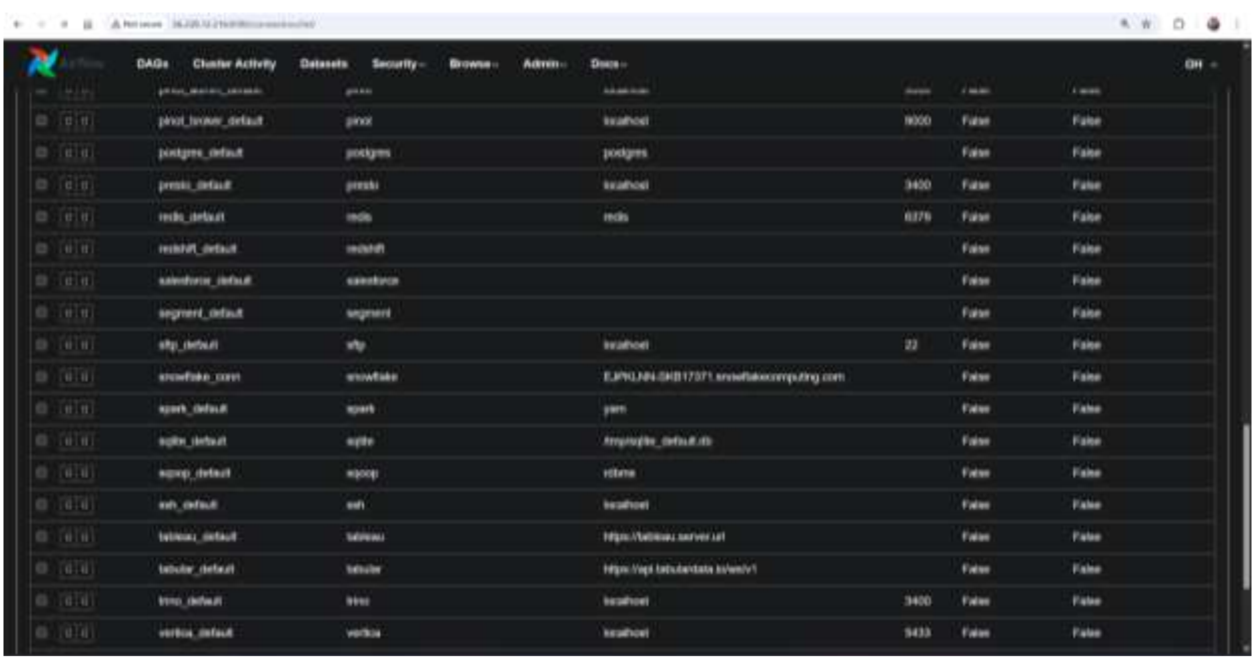
3 Enter these details:

- **Connection Id:** snowflake_conn
- **Connection Type:** Snowflake
- **Schema:** RAW
- **Login:** Aapka Snowflake username (e.g., TAHIRHASHMI90)
- **Password:** Aapka Snowflake password
- **Account:** EJPKLNN-SKB17371
- **Warehouse:** COMPUTE_WH
- **Database:** ECOMMERCE_DB
- **Region:** us-west-2 (Snowflake Region: Amazon (US West (Oregon))

In fields mein kuch likhne ki zaroorat nahi hai:

- **Description:** Ise khali chor den.
- **Port:** Snowflake k liye ise likhna zaroori nahi hota.
- **Path:** Ise bhi khali chor den.
- **Role:** Aapne Snowflake mein SYSADMIN dekha tha, lekin agar aap ise khali chor denge toh Airflow default role utha lega.
- **Private key (Path) & Private key (Text):** Inhein tab istemal karte hain jab hum password ke bajaye key-pair use karein. Aap password use kar rahe hain, isliye inhein khali chor dein.

Then, click on Save



STEP 2: DAGS FOLDER SETUP KAREIN

Ab humein wo jagah banani hai jahan hum apna Python automation code (DAG) rakhenge.
Apne **EC2 Terminal** (VS Code) par wapas jaen aur ye commands chalaein:

1. EC2 Terminal Login kr k activate kren.

```
ssh -i "<your-key>.pem" ec2-user@<your-public-ip>
source airflow_env/bin/activate
```

1. Airflow home mein dags folder banayein

```
mkdir -p ~/airflow/dags
```

2. Check karein ke folder ban gaya hai

```
ls ~/airflow
```

```
(airflow_env) [ec2-user@ip-172-31-28-66 ~]$ mkdir -p ~/airflow/dags
(airflow_env) [ec2-user@ip-172-31-28-66 ~]$ ls ~/airflow
airflow-webserver.pid  airflow.cfg  airflow.db  dags  logs  webserver_config.py
(airflow_env) [ec2-user@ip-172-31-28-66 ~]$
```

STEP 3: DIRECTED ACYCLIC GRAPH (DAG)

Ab hum aapki pehli **DAG (Directed Acyclic Graph)** file taiyar karenge jo Airflow ko batayegi ke usne Snowflake ke saath kya kaam karna hai.

1. File Create Karen

VS Code ke terminal mein (jahan airflow_env activated hai), ye command chalaein:

```
touch ~/airflow/dags/snowflake_automation.py
```

```
(airflow_env) [ec2-user@ip-172-31-27-195 ~]$ touch ~/airflow/dags/snowflake_automation.py
(airflow_env) [ec2-user@ip-172-31-27-195 ~]$ cd airflow
(airflow_env) [ec2-user@ip-172-31-27-195 airflow]$ cd dags
(airflow_env) [ec2-user@ip-172-31-27-195 dags]$ ls
snowflake_automation.py
```

Isse ek khali file ban jayegi. Ab ye command chala k file open kren.

```
nano ~/airflow/dags/snowflake_automation.py
```

2. DAG Code Paste Karein

```
from airflow import DAG
from airflow.providers.snowflake.operators.snowflake import SnowflakeOperator
from datetime import datetime, timedelta

# Default arguments
default_args = {
    'owner': 'Osama',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

# DAG Definition
with DAG(
    'snowflake_test_dag',
    default_args=default_args,
    description='A simple Snowflake automation DAG',
    schedule_interval='@hourly',  # Har ghante chalega
    catchup=False
) as dag:

    # Task: Snowflake se orders count check karna
    check_orders_count = SnowflakeOperator(
        task_id='check_orders_count',
        sql='SELECT COUNT(*) FROM ECOMMERCE_DB.RAW.ORDERS;',
        snowflake_conn_id='snowflake_conn', # Jo aapne Airflow UI mein banaya tha
    )

    check_orders_count
```

3. File Save aur Verify

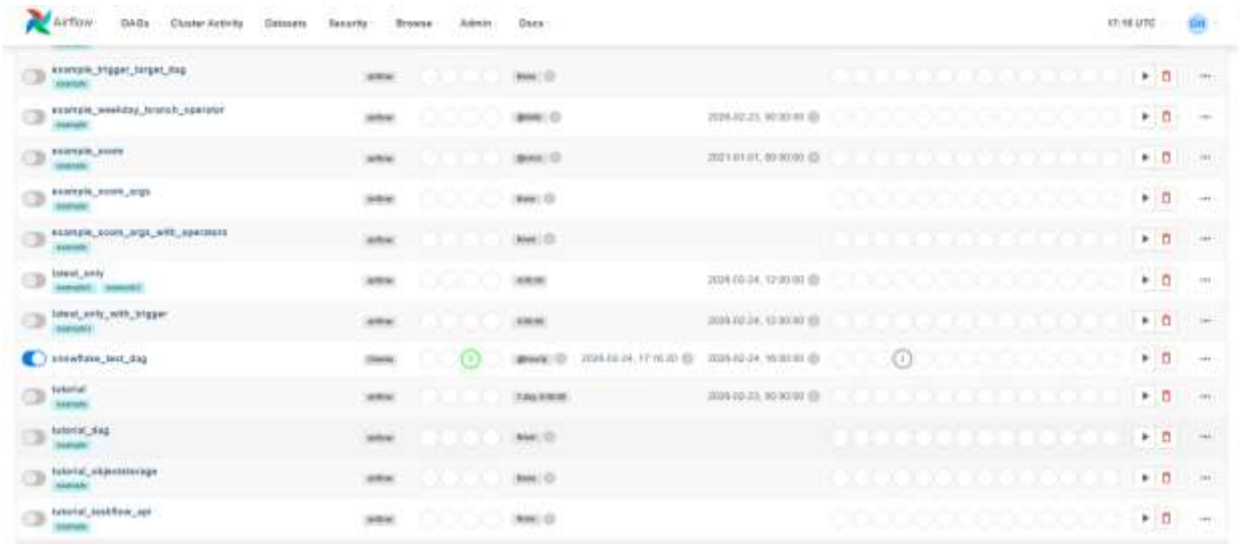
1. Code paste karne ke baad **Ctrl + 0**, then **Enter**, then **Ctrl + X**
2. Ab ye command chala kr check kren k Airflow ko ye new DAG mil gaya hai ya nhi:

```
airflow dags list
```

   - Agr list mein snowflake_test_dag nazar a rha hai, to apka kaam **100% perfect** hai!

4. Browser Refresh aur DAG Run

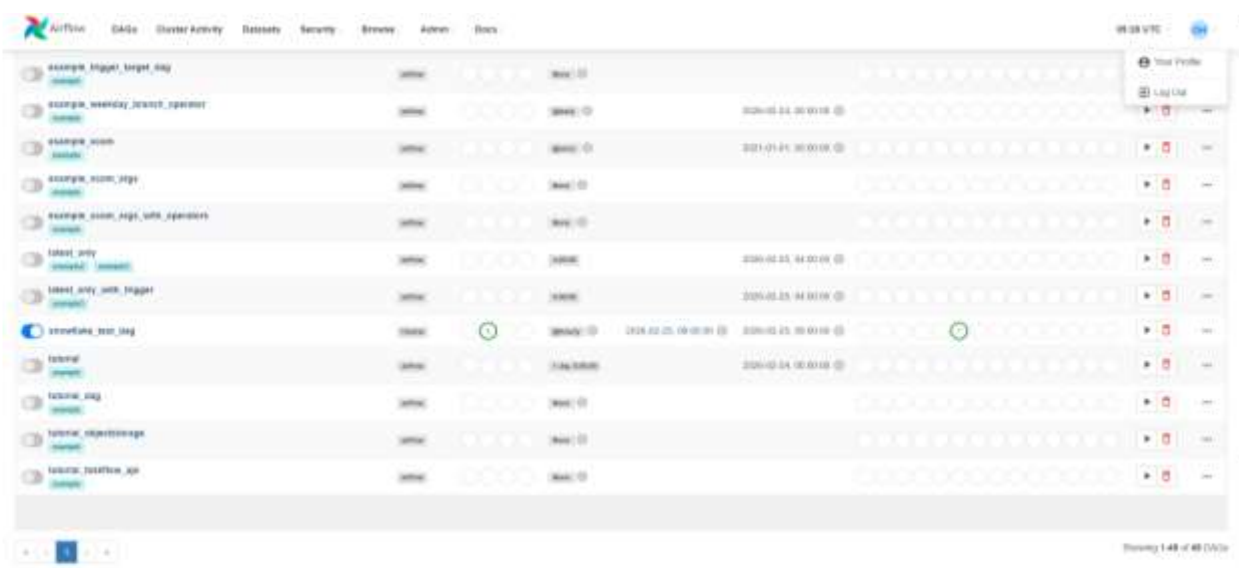
1) Browser (<your-public-ip>:8080) par wapas jaein aur Ctrl + F5 (Hard Refresh) karein.
2) snowflake_test_dag ke toggle switch ko ON karein.
3) Right side par Play (Trigger) button par click karein.



Agr aapka DAG queue mein hai aur chalne ki koshish kar raha hai means **Scheduler Band Hai,** ya aapka airflow scheduler terminal par crash kar gaya hai ya stop ho gaya hai, toh DAG kabhi bhi "Green" (Success) nahi hoga aur queue mein hi phasa rahega. To pehly Schedular start kren.

```
ssh -i "<your-key>.pem" ec2-user@<your-public-ip>

source airflow_env/bin/activate
export AIRFLOW_HOME=~/airflow
airflow scheduler -D
```

But,



Dark Green (success) matlab hai ke aapka Airflow ab Snowflake ke saath bilkul sahi tarah se connect ho raha hai aur usne data processing ki command kamyabi se poori kar li hai.

PHASE 4: DATA TRANSFORMATION

Aapka data.. **Raw** tables (ORDERS, CUSTOMERS) mein ja rha hai. In dono ko join kr k ek "Final Table" banana hi asli transformation hai.

Yahan do options hain, aap faisla karein:

Option 1: Manual Transformation

Agar aap check karna chahte hain ke join sahi ho raha hai, toh aap Snowflake mein ye query chala kar ek "Final Table" bana sakte hain:

```
CREATE OR REPLACE TABLE ECOMMERCE_DB.RAW.FINAL_SALES_REPORT AS
SELECT
  O.ORDER_ID,
  O.ITEM_NAME,
  O.PRICE,
  C.CUSTOMER_NAME,
  C.EMAIL,
  C.COUNTRY
FROM ECOMMERCE_DB.RAW.ORDERS O
JOIN ECOMMERCE_DB.RAW.CUSTOMERS C
 ON O.CUSTOMER_NAME = C.CUSTOMER_NAME;
```

Iska faida ye hai ke aapko foran result dikh jayega.

Option 2: Airflow Transformation (Recommended)

Asli pipeline mein hum chahte hain ke ye join **khud-ba-khud** har 5 ya 10 minute baad chale. Airflow se transformation karne ke ye faide hain:

1. **Automation:** Aap ko baar-baar query nahi chalani paregi.
2. **Handling Late Data:** Kabhi orders pehle aa jate hain aur customers baad mein. Airflow in cheezon ko handle karta hai.
3. **Cleanliness:** Aap ki Raw tables gandi nahi hongi, sirf final table clean hogi.

My Opinion:

Aap ne Snowpipe se data load toh kar liya hai, ab Direct Airflow par chaliye**.** Hum Airflow mein ek "Task" likhenge jo Snowflake ko order dega: *"Bhai, pichle 5 minute ka data join kar ke report table mein daal do."* Is se aapka poora End-to-End Pipeline complete ho jayega.

Transformation *hi woh stage hai jahan ek Data Engineer "Raw Data" (kachra) ko "Clean Data" (sona) mein badalta hai.*

*Abhi aapka data S3 se seedha ORDERS table mein ja raha hai. Agar EC2 se ek hi file do baar load ho jaye, toh Snowflake mein duplicates aa jayenge. Hum Airflow ke zariye ek **Idempotent Pipeline** banayenge... yani aisi pipeline jo jitni baar bhi chale, data ko kabhi kharab ya duplicate nahi karegi.*

Plan: "The Gold Layer Transformation"

Hum ek nayi table banayenge ANALYTICS_ORDERS aur Airflow mein ek naya task add karenge jo:

1. RAW.ORDERS se data uthayega.
2. Duplicates khatam karega.
3. Sirf unique records ko ANALYTICS_ORDERS mein dalega.

Step 1: Snowflake mein Analytics Table Banaen

Pehle Snowflake Worksheet mein ja kar ye table banayein:

```
USE DATABASE ECOMMERCE_DB;
USE SCHEMA RAW;

CREATE OR REPLACE TABLE ECOMMERCE_DB.RAW.ANALYTICS_ORDERS (
   ORDER_ID NUMBER,
   CUSTOMER_NAME STRING,
   ITEM_NAME STRING,
   PRICE NUMBER,
   INSERTED_AT TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
);
```

Step 2: DAG Code Update Karen

Ab apne EC2 terminal par jayein aur purani DAG file ko edit karein:

```
nano ~/airflow/dags/snowflake_automation.py
```

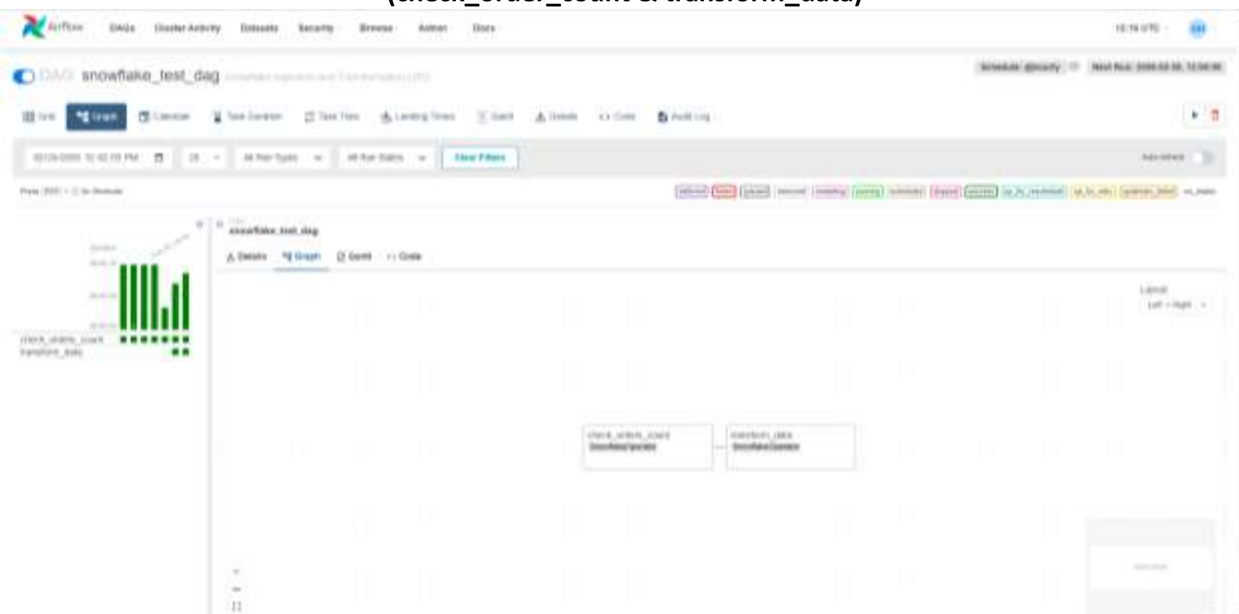
Purane code ko hata kar ye naya code paste karein (is mein humne **Transformation Task** add kiya hai)

```
from airflow import DAG
from airflow.providers.snowflake.operators.snowflake import SnowflakeOperator
from datetime import datetime, timedelta

# Default arguments
default_args = {
    'owner': 'Osama',
    'depends_on_past': False,
    'start_date': datetime(2024, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

# DAG Definition
with DAG(
    'snowflake_test_dag',
    default_args=default_args,
    description='Snowflake Ingestion and Transformation DAG with Deduplication',
    schedule_interval='@hourly',
    catchup=False
) as dag:

# Task: Transform & Clean Data (Strict Deduplication on ORDER_ID)
# Ye query sirf har ORDER_ID ka pehla record uthayegi aur duplicates delete kar degi
transform_data = SnowflakeOperator(
    task_id='transform_data',
    sql="""
    INSERT OVERWRITE INTO ECOMMERCE_DB.RAW.ANALYTICS_ORDERS (
        ORDER_ID,
        CUSTOMER_NAME,
        ITEM_NAME,
        PRICE
    )
    SELECT
        ORDER_ID,
        CUSTOMER_NAME,
        ITEM_NAME,
        PRICE
    FROM ECOMMERCE_DB.RAW.ORDERS
    QUALIFY ROW_NUMBER() OVER (PARTITION BY ORDER_ID ORDER BY ORDER_ID) = 1;
    """,
    snowflake_conn_id='snowflake_conn',
)

# Task Dependency
    check_orders_count >> transform_data
```

Step 3: File Save aur Run Karein

1. **Save:** Ctrl + O, Enter, Ctrl + X.
2. **Refresh Dashboard:** Airflow dashboard par ja kar page refresh karein. Aapko ab apne DAG (snowflake_test_dag) ke andar Graph mein **2 boxes** nazar aayenge.

(check_order_count & transform_data)

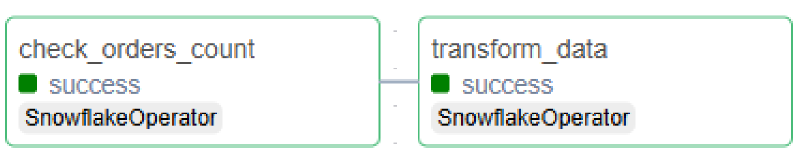


3. **Trigger:** Play button dabaen. (Top right)

Purpose:

Aapne Airflow ko bataya: "Pehle count check karo (check_orders_count), aur agar woh kamyabi se ho jaye, toh phir transformation shuru karo (transform_data)." Is query mein humne **INSERT OVERWRITE** aur **DISTINCT** use kiya hai. Iska faida ye hai ke agar RAW.ORDERS mein 100 duplicate entries bhi hon, toh aapki ANALYTICS_ORDERS table hamesha saaf-suthri rahegi.

Final Check



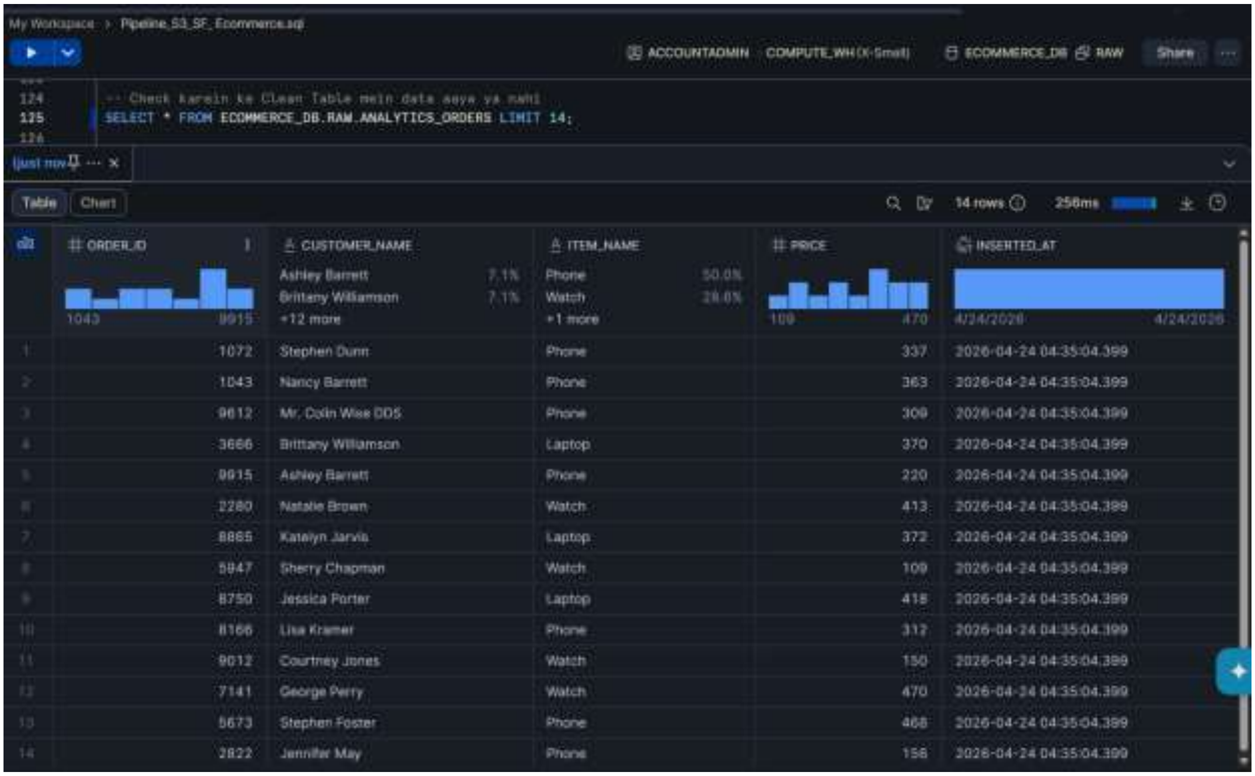
Congratulations! Airflow mein dono tasks **Dark Green (Success)** hogye hain.

- **Orchestration Success:** Airflow ne pehle Snowflake se contact kiya, data check kiya, aur phir clean data ko move karne ki command di. Sab kuch automated tarike se bina kisi manual intervention ke hua.

- **Pipeline Integration:** Aapki poori chain
  (EC2 -> Kinesis -> S3 -> Snowpipe -> Airflow -> Snowflake Tables) ab live aur working hai.

Snowflake mein ye command chala k check kren k aap k ANALYTICS_ORDERS table mein Clean data aya ya nahi.

```
SELECT * FROM ECOMMERCE_DB.RAW.ANALYTICS_ORDERS LIMIT 14;
```



Mubarak ho! Aapka Data Analytics_Orders table mein dikh rha hai.

Check karein ke duplicates khatam hue ya nahi (Count km hona chahiye agar duplicates the)

```
SELECT
    (SELECT COUNT(*) FROM ECOMMERCE_DB.RAW.ORDERS) AS RAW_ORDERS_COUNT,
    (SELECT COUNT(*) FROM ECOMMERCE_DB.RAW.ANALYTICS_ORDERS) AS ANALYTICS_ORDERS_COUNT,
    (SELECT COUNT(*) FROM ECOMMERCE_DB.RAW.ORDERS) - (SELECT COUNT(*) FROM ECOMMERCE_DB.RAW.ANALYTICS_ORDERS) AS DUPLICATES_REMOVED;
```

Mubarak ho! Apne ab sirf data load nahi kiya, blke ek **Data Transformation Layer** bhi bana li hai.

1. Unique vs Total Count Check or Duplicate Check

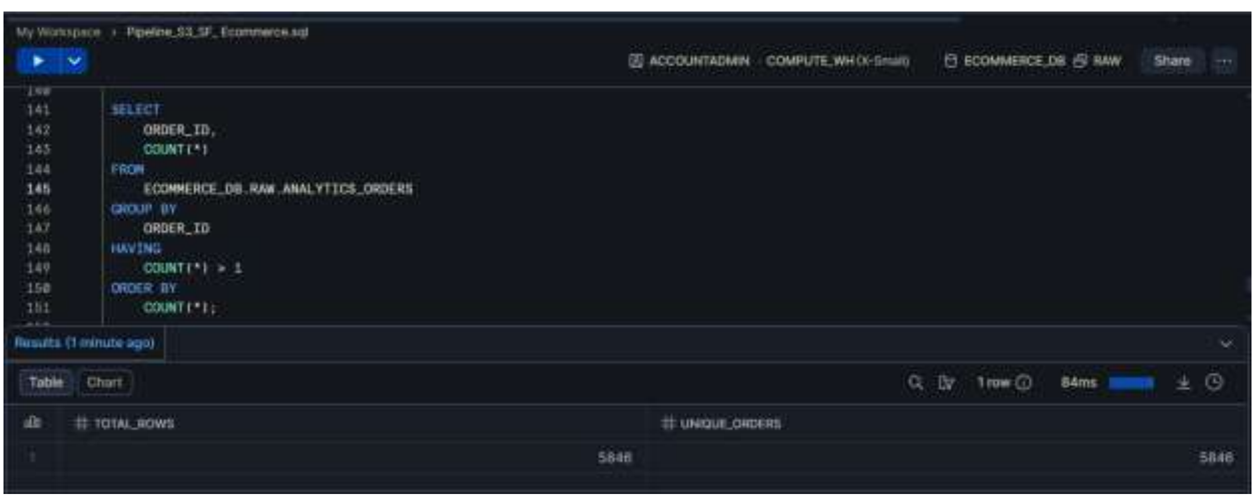
- Ye query apko bataegi k total lines kitni hain aur unique ORDER_ID kitne hain.
- Agr dono numbers barabar hain, toh koi duplicate nahi hai.

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT ORDER_ID) AS unique_orders
FROM ECOMMERCE_DB.RAW.ANALYTICS_ORDERS;
```

2. Duplicates Ki List Nikalna

Agar aapko shak hai, toh ye query chalaein. Agar iska result **khali (empty)** aata hai, toh iska matlab hai ke aapka data 100% saaf hai:

```
SELECT ORDER_ID, COUNT(*)
FROM ECOMMERCE_DB.RAW.ANALYTICS_ORDERS
GROUP BY ORDER_ID
HAVING COUNT(*) > 1;
```



DBT (DATA BUILD TOOL)

dbt (data build tool) ko is project mein shamil karne ka matlab hai ke aap "Basic Data Engineering" se nikal kar **"Analytics Engineering"** ki professional duniya mein Qadam rakh rahe hain.

Abhi tak aapne Airflow ke andar SQL queries likhi hain, lekin **dbt** aapko SQL ko as a "software code" manage karne ki taqat deta hai (with version control, testing, aur documentation).

---

dbt Kyun Use Karein? (The "Pro" Advantage)

Feature	Airflow SQL (Jo abhi hai)	dbt (Jo aap karenge)
Testing	Manual check karni parti hai.	dbt test se unique aur null values automatic check hoti hain.
Documentation	Kuch nahi hota.	dbt docs generate se poora lineage graph ban jata hai.
Version Control	Sirf DAG file ka hota hai.	Har model (table) ka alag version control hota hai.
Lineage	Mushkil se samajh aata hai.	Visual map milta hai ke data kahan se kahan ja raha hai.

Aapka Action Plan:

1. **Snowflake Setup:** dbt ke liye Snowflake mein ek alag DBT_ROLE aur DBT_USER banana best practice hai (taaki security maintain rahe).
2. **Architecture Update:** Aapka flow ab ye hoga:

EC2 (Generator) -> Kinesis -> S3 -> Snowpipe -> Snowflake (Raw) -> dbt (Transformation) -> Snowflake (Analytics) -> QuickSight.

Osama, kya aap dbt Core (Terminal) use karna chahte hain ya dbt Cloud (Web Interface)? (Main mashwara dbt Core ka dunga kyunke aap EC2 par pehle se setup hain).

Step 1: Integration of dbt

Ab aapko ye steps follow karne chahiye:

1. RAW & ANALYTICS Schema

Snowflake mein ANALYTICS schema banaen, jis mein Transformed data ae.

```
CREATE OR REPLACE SCHEMA ANALYTICS;
USE SCHEMA ANALYTICS;
```

Snowflake mein Permissions Fix Karein

```
USE ROLE ACCOUNTADMIN;

-- Role ko database aur schema ki usage dena
GRANT USAGE ON DATABASE ECOMMERCE_DB TO ROLE SYSADMIN;
GRANT USAGE ON SCHEMA ECOMMERCE_DB.ANALYTICS TO ROLE SYSADMIN;

-- Schema mein tables banane ki ijazat dena
GRANT CREATE TABLE ON SCHEMA ECOMMERCE_DB.ANALYTICS TO ROLE SYSADMIN;

-- Agar pehle se koi tables hain toh un par bhi rights dena
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA ECOMMERCE_DB.ANALYTICS TO ROLE SYSADMIN;
```

2. dbt Setup (Installation and Initialization)

Aapko apne EC2 instance par dbt install karna hoga. Kyunke aap Python par kaam kar rahe hain, aap dbt-snowflake adapter use karenge:

- **Installation:** `pip install dbt-snowflake`
- **Initialize:** `dbt init ecommerce_dbt`

  Iske baad aapko `profiles.yml` file mein Snowflake ke credentials (account URL, role, warehouse etc) dene honge.

  - **Database:** 1 (Snowflake select karein)
  - **Account:** Snowflake account identifier (EJPKLNN-SKB17371).
  - **User:** Apni Username (TAHIRHASHMI90)
  - **Password:** Apna Snowflake password (12qwaszx9090Osama)
  - **Role:** SYSADMIN
  - **Warehouse:** COMPUTE_WH
  - **Database:** ECOMMERCE_DB
  - **Schema:** ANALYTICS (Kyunke dbt hamara saaf data yahan banayega).
  - **Threads:** 1

2. Connection Test (dbt debug)

Terminal mein apne project folder ke andar ja kar ye command chalaein:

```
cd ecommerce_dbt

dbt debug
```

Agar ye command **"Connection test: [OK connection ok]"** dikha rha hai. Iska matlab hai k **dbt** aur **Snowflake** k darmiyan dosti pakki ho chuki hai aur ***dbt*** ab ***Snowflake*** par query chala sakta hai.

3. Model Development (Transformation Logic)

Jab aap dbt init karte hain, toh dbt ne aapke liye kuch folders banaye hain. Inka maqsad samajhna zaroori hai:

- models/: Yahan aap apni SQL transformations likhenge.
- dbt_project.yml: Ye aapke poore project ki main settings file hai.
- target/: Jab aap dbt run karte hain, toh final compiled SQL yahan banti hai.

Step 1: dbt ko Source Table batana (sources.yml)

dbt ko ye batana parta hai ke "Raw" data kahan para hai. Iske liye models folder mein ek nayi file banayein:

```
nano models/sources.yml
```

Usme ye code paste karein:

```
version: 2

sources:
  - name: raw_data
    database: ECOMMERCE_DB
    schema: RAW
    tables:
      - name: orders
        columns:
          - name: order_id
            tests:
              - not_null
              - unique
```

Step 2: Pehla dbt Model Banana (stg_orders.sql)

Ab hum wahi logic likhenge jo humne Airflow mein verify kiya tha, lekin dbt ke andaz mein.

```
nano models/stg_orders.sql
```

Usme ye code paste karein:

```
{{ config(materialized='table') }}

with raw_orders as (
   select * from {{ source('raw_data', 'orders') }}
)

select
   order_id,
   customer_name,
   item_name,
   price
from raw_orders
qualify row_number() over (partition by order_id order by order_id) = 1
```

Step 3: dbt Run Karen

Ab terminal par ye command chalaein:

```
dbt run
```

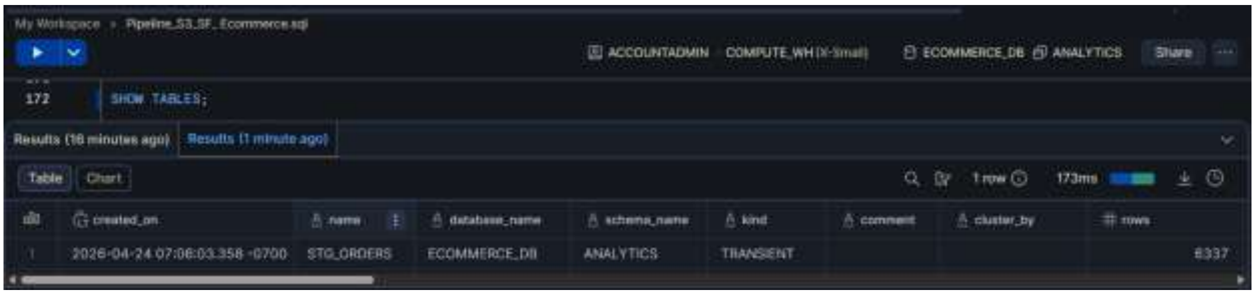
"Completed successfully". Aapne dbt ka pehla model (stg_orders) kamyabi se run kar liya hai aur usne Snowflake mein table create kar di hai.

Yahan tak aapne aik mukammal **Modern Data Stack** ka basic structure khara kar liya hai:

1. **Airflow:** Jo data pipeline ko automate kar raha hai.
2. **Snowflake:** Jahan aapka raw aur analytics data store ho raha hai.
3. **dbt:** Jo complex transformations aur cleaning (deduplication) handle kar raha hai.

Isse Kya Hoga?

1. **Deduplication:** dbt khud hi ANALYTICS schema mein stg_orders ke naam se ek table banayega.
2. **Cleaning:** Ye table sirf unique Order IDs rakhega.
3. **Professionalism:** Ab aapka Airflow DAG sirf dbt run ki ek command chalayega, aur saari complex cleaning dbt handle karega.



STG_ORDERS table mein data ki uniqueness confirm karne ke liye aapke paas do behtareen tareeqay hain: ek dbt ka internal testing feature aur dusra Snowflake ki manual query.

1. dbt Tests (The Professional Way)

dbt ka sabse bada faida hi ye hai ke wo data quality ko automatically check karta hai. Aapne jo models/sources.yml mein unique aur not_null ke tests add kiye hain, unhein run karke aap 100% confirm kar sakte hain.

Terminal mein ye command chalaein:

```
dbt test
```

Aapke dbt test ka result bilkul wahi dikha raha hai jiski hum umeed kar rahe thay!

Iska Matlab Kya Hai?

- **PASS (not_null):** Iska matlab hai ke aapke data mein koi bhi order_id khaali (null) nahi hai.
- **FAIL (unique):** dbt ne bataya ke source.raw_data.orders mein 3118 aise records hain jo repeat ho rahe hain.

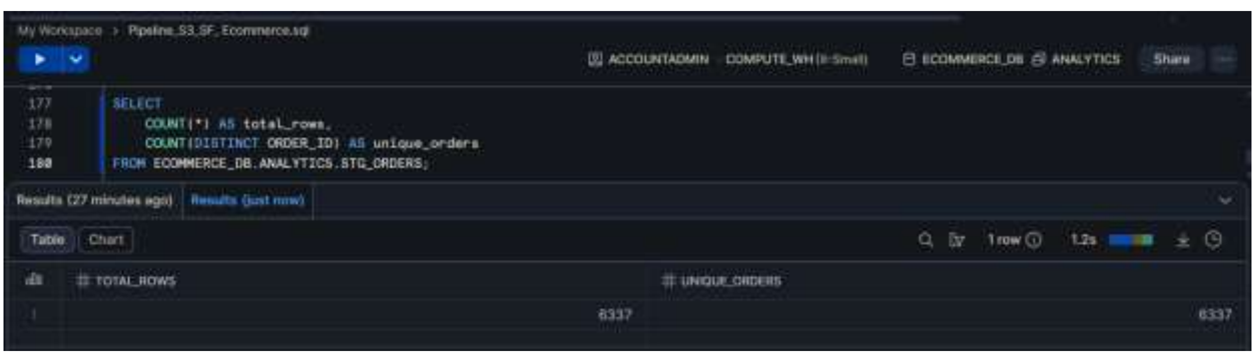
Ye "Fail" hona asal mein aapki Jeet hai! Because:

1. Isne confirm kar diya ke aapka **Raw data ganda tha**.
2. Lekin humne Snowflake mein pehle hi verify kiya hai ke aapka **STG_ORDERS (clean table) bilkul sahi hai** (6337 rows).

2. Snowflake Manual Check (The Direct Way)

Agar aap apni aankhon se Snowflake mein result dekhna chahte hain, toh Snowflake worksheet mein ye query chalaein:

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(DISTINCT ORDER_ID) AS unique_orders
FROM ECOMMERCE_DB.ANALYTICS.STG_ORDERS;
```



POWER OF dbt (Remove the Errors):

Ab hum ye test apne **Clean Table (stg_orders)** par chalayenge taaki dbt humein **"Green/PASS"** signal de. Iske liye models/schema.yml file banayein

```
nano models/schema.yml
```

ye code dalen:

```
version: 2

models:
  - name: stg_orders
    columns:
      - name: order_id
        tests:
          - unique
          - not_null
```

Iske baad terminal par ye command chalaein:

```
dbt test --select stg_orders
```

Ye test **100% PASS** hona chahiye kyunke aapki transformation logic (QUALIFY ROW_NUMBER()) ne duplicates pehle hi khatam kar diye hain.

```
(airflow_env) [ec2-user@ip-172-31-21-217 ecommerce_dbt]$ nano models/schema.yml
(airflow_env) [ec2-user@ip-172-31-21-217 ecommerce_dbt]$ dbt test --select stg_orders
/home/ec2-user/airflow_env/lib64/python3.9/site-packages/networkx/utils/backends.py:135: RuntimeWarning: networkx backend defined more than once: nx-loopback
  backends.update(_get_backends("networkx.backends"))
16:13:57  Running with dbt=1.11.0-b3
16:13:58  Registered adapter: snowflake=1.10.2
16:13:59  [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.ecommerce_dbt.example
16:13:59  Found 1 model, 4 data tests, 1 source, 492 macros
16:13:59
16:13:59  Concurrency: 1 threads (target='dev')
16:13:59
/home/ec2-user/airflow_env/lib64/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026. To continue r
eceiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-s
upport-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
16:14:01  1 of 2 START test not_null_stg_orders_order_id .................................. [RUN]
/home/ec2-user/airflow_env/lib64/python3.9/site-packages/boto3/compat.py:89: PythonDeprecationWarning: Boto3 will no longer support Python 3.9 starting April 29, 2026. To continue r
eceiving service updates, bug fixes, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-s
upport-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
16:14:04  1 of 2 PASS not_null_stg_orders_order_id ........................................ [PASS in 2.96s]
16:14:04  2 of 2 START test unique_stg_orders_order_id .................................... [RUN]
16:14:04  2 of 2 PASS unique_stg_orders_order_id .......................................... [PASS in 0.64s]
16:14:05
16:14:05  Finished running 2 data tests in 0 hours 0 minutes and 6.16 seconds (6.16s).
16:14:05
16:14:05  Completed successfully
16:14:05
16:14:05  Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
(airflow_env) [ec2-user@ip-172-31-21-217 ecommerce_dbt]$
```

Mubarak ho! jiski hume talash thi who nzr a rha hai: **"Completed successfully"** and **"PASS=2"** Aapne dbt ki aslee taqat ka muzahira dekh liya hai:

- **not_null_stg_orders_order_id: PASS** — Iska matlab hai koi Order ID missing nahi hai.
- **unique_stg_orders_order_id: PASS** — Iska matlab hai aapki transformation ne duplicates ko 100% khatam kar diya hai.

Step 2: Airflow + dbt Automation (The Final Link)

Ab humein Airflow ko batana hai ke Snowflake mein data load karne ke baad wo khud hi dbt ke tests aur models run kare.

1. Airflow Environment mein Bash Commands check karein

Kyunke dbt airflow_env mein install hai, Airflow ka BashOperator ise asani se chala sakta hai.

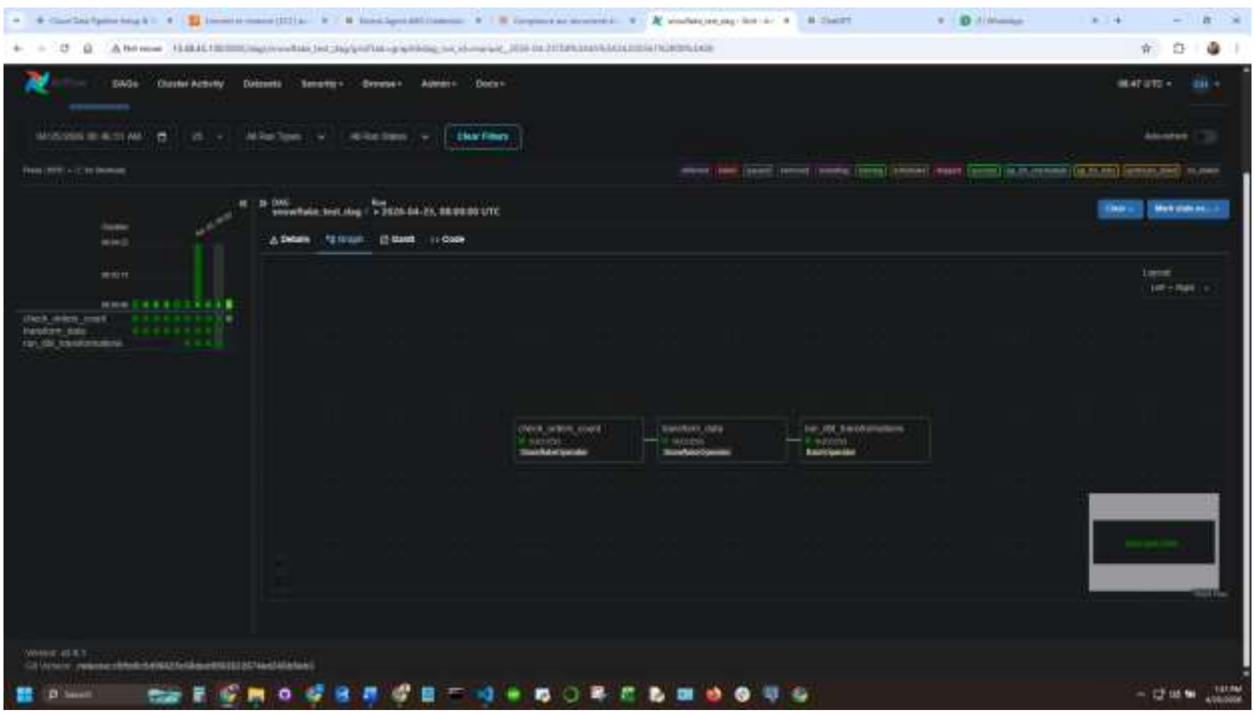
2. DAG File Update Karein

```
cd ..

nano ~/airflow/dags/snowflake_automation.py
```

Apni Airflow DAG file mein BashOperator ke zariye ye do naye tasks add karein:

```
from airflow import DAG
from airflow.providers.snowflake.operators.snowflake import SnowflakeOperator
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'start_date': datetime(2026, 4, 24),
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    'snowflake_test_dag',
    default_args=default_args,
    schedule_interval='@hourly',
    catchup=False,
) as dag:

    # 1. Raw count check
    check_orders_count = SnowflakeOperator(
        task_id='check_orders_count',
        snowflake_conn_id='snowflake_conn',
        sql="SELECT COUNT(*) FROM ECOMMERCE_DB.RAW.ORDERS;"
    )

    # 2. Snowflake transformation
    transform_data = SnowflakeOperator(
        task_id='transform_data',
        snowflake_conn_id='snowflake_conn',
        sql="""
            CREATE OR REPLACE TABLE ECOMMERCE_DB.RAW.ANALYTICS_ORDERS AS
            SELECT * FROM ECOMMERCE_DB.RAW.ORDERS
```

```
        QUALIFY ROW_NUMBER() OVER (PARTITION BY ORDER_ID ORDER BY ORDER_ID) = 1;
    """
)

# 3. dbt run (Cleaned Models)
# Yahan path wahi use kiya hai jo aapne terminal mein check kiya
run_dbt_transformations = BashOperator(
    task_id='run_dbt_transformations',
    bash_command='cd /home/ec2-user/ecommerce_dbt && /home/ec2-user/airflow_env/bin/dbt run',
)

check_orders_count >> transform_data >> run_dbt_transformations
```



Phase 5: Data Visualization (Amazon QuickSight)

Ab jabki humara clean aur transformed data Snowflake ki `ANALYTICS_ORDERS` table mein maujood hai, hume ise visualize karna hai taaki business decision-making ki ja sake. Iske liye hum **Amazon QuickSight** ka istemal karenge.

Step 1: QuickSight setup aur Snowflake Connection

Sabse pehle hume QuickSight ko Snowflake se connect karna hoga taaki woh wahan se data fetch kar sake.

1. **QuickSight Console** par jayen aur 'Manage Data' -> 'New Dataset' par click karein.
2. Data source ki list se **Snowflake** select karein.
3. **Connection Details** bharein:
    - **Data Source Name:** Snowflake_Ecommerce_DS
    - **Database Server:** (Apna Snowflake account URL likhein)
    - **Warehouse:** COMPUTE_WH
    - **Database:** ECOMMERCE_DB
    - **Schema:** RAW (ya jo bhi aapne analytics ke liye banaya hai)
    - **Username/Password:** Apne Snowflake credentials enter karein.
4. 'Create data source' par click karein.

Step 2: Dataset taiyar karna

Connection banne ke baad hume specific table select karni hai.

1. Tables ki list se ANALYTICS_ORDERS table ko select karein.
2. **Edit/Preview Data** par click karein taaki aap columns ke data types (Date, String, Number) check kar sakein.
3. Data ko **SPICE** (QuickSight’s in-memory engine) mein import karein taaki dashboard fast chale.

Step 3: Visuals aur Dashboard Design

Ab hum different charts banayenge jo ecommerce performance dikhayenge:

1. **Total Sales (KPI Card):** SUM(ORDER_AMOUNT) ka use karke total revenue dikhayen.
2. **Order Trends (Line Chart):** ORDER_DATE ko X-axis par aur COUNT(ORDER_ID) ko Y-axis par rakh kar daily/monthly orders ki growth check karein.
3. **Category Distribution (Pie Chart):** Ye dekhne ke liye ke kaunsi category ke products sabse zyada bik rahe hain.
4. **Top Customers (Table/Bar Chart):** CUSTOMER_ID ke hisaab se highest spending dikhayen.

Step 4: Automation (Refresh Schedule)

Q k humara Airflow pipeline hourly chalta hai, toh hume QuickSight ko bhi schedule karna hoga:

- QuickSight dataset settings mein ja kar **Schedule Refresh** set karein taaki har ghante Snowflake se naya data fetch ho sake.
